

Java Programming

Department of Information Technology

(3rd YEAR – 1st SEM)

2026-2027

Prepared by:

Dr Samia Souror

Table of contents

Table of contents ..... **i**

Chapter 1: Introduction to Database **4**

1.1 Introduction 4

1.2 Overview of Database Technology 5

1.3 Data 5

1.4 Database 6

1.5 Database Management System 8

1.6 Advantages of a DBMS 9

1.7 Disadvantages of a DBMS 11

1.8 Benefits of Database Systems 12

1.9 Purpose of Database Systems 14

1.10 Database Architecture 20

1.10.1 Main Components of a Database System 21

1.10.2 Database Application Architectures 22

1.11 Why Database Systems Were Developed 23

1.12 View of Data 23

1.13 Data Abstraction 24

1.14 Instances and Schemas 25

1.15 Data Models 26

1.15.1 Relational Model 26

1.15.2 Entity-Relationship (E-R) Model 26

1.15.3 Object-Based Data Model 26

1.15.4 Semi-Structured Data Model 27

1.16 Database Languages 27

1.16.1 Data-Definition Language (DDL) 27

1.16.2 Data-Definition Language (DDL) 28
1.17 Data Dictionary 29
1.17.1 Functions of a Data Dictionary 30
1.18 Database Administrators and Database Users 31
1.18.1 Database Users and User Interfaces 31
1.19 Query Processor 32
1.19.1 Storage Manager 33
1.19.2 Transaction Manager 34
Chapter 2: Entity Relationship Diagram (ERD) 40
2.1 Introduction 40
2.2 Database Design Techniques 42
2.2.1 Key Components of an ERD 43
2.3 Advantages of ER Modeling 50
2.4 Disadvantages of ER Modeling 50
2.5 Relational Model 50
2.6 Domains and Atomic Values 54
2.7 Null Values 54
2.8 Database Schema 54
2.9 Keys 57
2.10 Referential Integrity 59
2.11 Schema Diagrams 59
Chapter 3: The Structure Query Language (SQL) 67
2.1 Introduction 67
2.2 Important Features of SQL 69
2.3 Data Definition Language (DDL) 70
2.4 Data Manipulation Language (DML) 75
Chapter 4: Data Control 87

4.1 Introduction 87

4.2 DATABASE OBJECTS: VIEWS, SEQUENCES, ... 89

INDEXES AND SYNONYMS 89

Chapter 1: Introduction to Database

1.1 Introduction

Data is one of the most important resources in modern organizations. Universities, banks, hospitals, companies, government institutions, e-commerce platforms, and social media applications all generate and depend on large amounts of data. This data may include information about students, employees, customers, products, transactions, courses, patients, accounts, and many other entities.

In the early days of computing, organizations commonly stored data in separate files. Although file-based systems were useful for simple applications, they became difficult to manage as the volume and complexity of data increased. Problems such as data duplication, inconsistent information, difficulty in sharing data, limited security, and complicated data retrieval became increasingly common.

Database systems were developed to overcome many of these limitations. A database provides an organized collection of related data, while a Database Management System (DBMS) provides the software tools required to create, store, organize, retrieve, update, and manage that data. Today, database systems form the foundation of almost every information system. Understanding database concepts is therefore an essential skill for students of computer science, information technology, software engineering, and related disciplines.

This chapter introduces the fundamental concepts of databases and database management systems. It discusses the evolution from traditional file-processing systems to database systems, the main characteristics of databases, the components of a database

environment, the advantages and limitations of DBMSs, database users, and common database applications.

1.2 Overview of Database Technology

Database technology is an effective technology used for data management. It studies how to manage data scientifically so as to provide people with shareable, secure and reliable data. It involves four important concepts, as shown in Fig. 1.1, which are introduced below.

1.3 Data

Data refers to the raw records that have not been processed. Generally speaking, data is not clearly organized and classified, and thus cannot clearly express the meaning of what things represent. Data can be a pile of magazines, a stack of newspapers, minutes of a meeting, or a copy of medical records. Early computer systems were primarily used for scientific calculations and dealt with numerical data, that is, numbers in the generalized concept of data, such as integers like 1, 2, 3, 4, 5, but also floating point numbers like 3.14, 100.34, and A25.336.

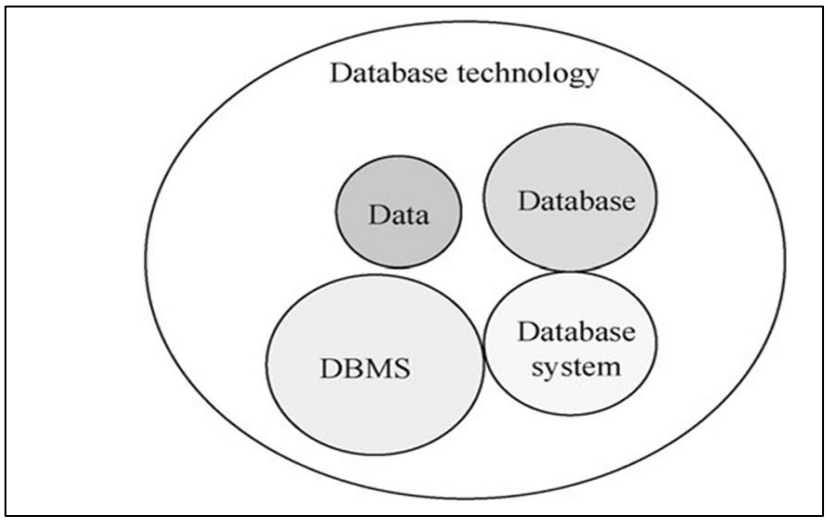


Figure 1.1: Overview of database technology

In modern computing environments, the concept of **data** encompasses a wide range of digital representations. Data may take the form of numbers, text, graphics, images, audio, video, and many other types of content. Once these forms of data are converted into digital formats, they can be stored, processed, and managed by computer systems, as illustrated in Figure 1.2.

Data is not limited to its physical or digital representation; it also has **semantics**, which refers to the meaning and significance associated with the data. The meaning of a particular data value depends largely on the context in which it is used. Therefore, the same data value can represent completely different information in different situations.

For example, the value **88** may represent the total number of employees in a department, a student's score in a particular course, the price of a product, or a person's weight in kilograms. Although the value itself remains the same, its meaning changes according to the context. This demonstrates why understanding the semantics of data is essential when designing and managing database systems.

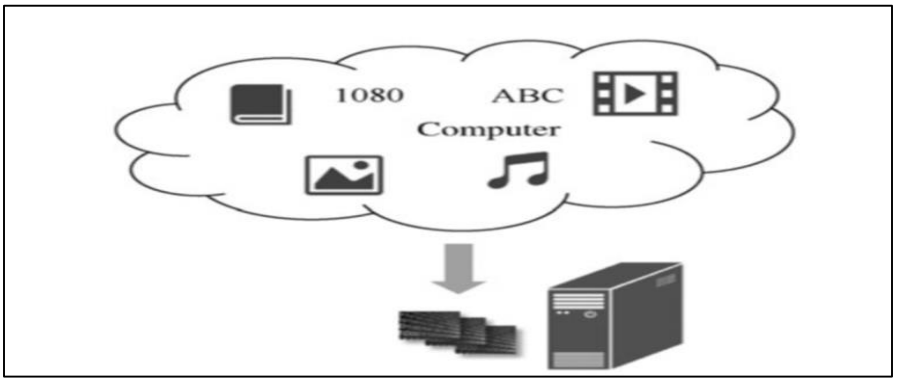


Figure 1.2: the Processing Information

1.4 Database

A database should provide several fundamental characteristics to ensure that data can be stored, managed, and accessed effectively.

- **Long-Term Storage:** A database should provide a reliable mechanism for storing data over an extended period. The system should also support backup and recovery mechanisms so that data can be restored after hardware failures, software errors, or other unexpected incidents, thereby minimizing the risk of data loss.
- **Organization:** Data should be systematically organized, described, and stored according to an appropriate data model. Proper organization helps reduce unnecessary data duplication, improves data independence, and makes the database easier to maintain, modify, and expand as requirements change.
- **Data Sharing:** Data stored in a database should be accessible to multiple authorized users and applications rather than being restricted to a single user. A database management system should support concurrent access while maintaining data consistency and security. For example, a *student information database*, as illustrated in Figure 1.3, may need to be accessed simultaneously by different categories of users, including students, instructors, and parents. Each user may have different access privileges depending on their role. Students may view their academic records, instructors may manage grades and course information, while parents may be permitted to view selected student information.

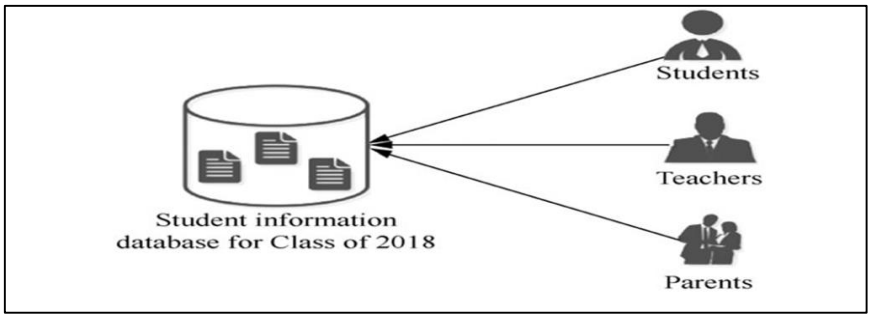


Figure 1.3 Database

1.5 Database Management System

A Database System (DBS) is an integrated system designed to store, manage, process, and maintain data efficiently. It consists of the database, Database Management System (DBMS), application development tools, database applications, and database administrators (DBAs). As illustrated in Fig. 1.4, all components shown in the figure, except the user and the operating system, constitute the main components of the database system. Database management system (DBMS) is a system software located between the user and the operating system that can organize and store data scientifically, access and maintain data efficiently. Like the operating system, the DBMS is also the basic software of the computer system.

The DBMS mainly functions as follows.

(1) **Data definition:** The DBMS provides data definition language (DDL), through which the user can easily define the composition and structure of data objects in the database.

(2) **Data organization, storage and management:** The DBMS is responsible for organizing, storing and managing data in a classified manner, involving data dictionaries, user data, data access paths, etc. The DBMS also has to determine in which file structure and which access method to organize this data in the storage space, and how to realize the linkage between the data. The basic goal of data organization and storage is to improve storage space utilization, facilitate data access, and provide a variety of data access to improve access efficiency.

(3) **Data manipulation:** DBMS also provides data manipulation language (DML), with which users can manipulate data to achieve such basic operations as query, insert, deletion and modification on data.

(4) **Transaction management and operation management of the database:** The data- base is unified managed and controlled by the DBMS during establishment, operation and maintenance to ensure the correct operation of transactions, the security and integrity of data, the concurrent use of data by multiple users and the system recovery after a failure.

(5) **Database establishment and maintenance:** This function covers the database initial data input and conversion, database dump and recovery, database reorganization and performance monitoring, analysis function, etc. These functions are usually implemented by certain programs or management tools.

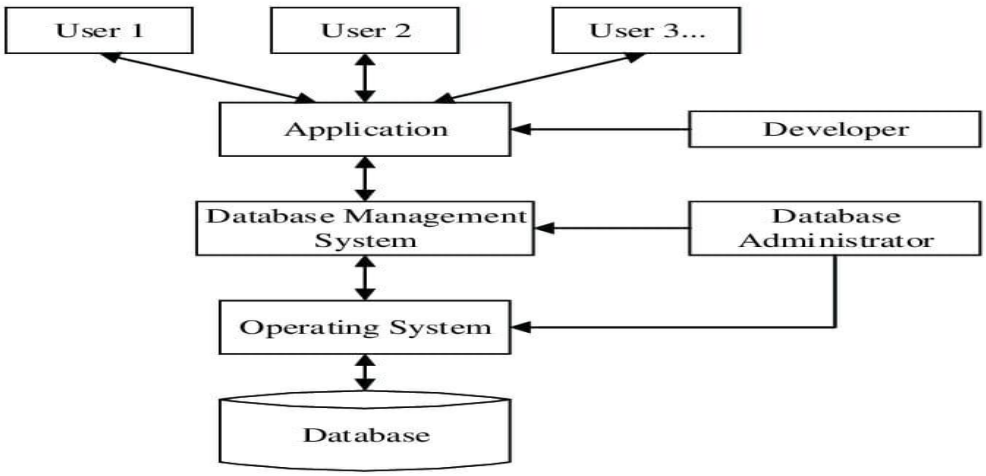


Figure 1.4 Database System

1.6 Advantages of a DBMS

1) **Control of Data Redundancy:** Data redundancy refers to the unnecessary duplication of the same data in multiple locations. In a database system, centralized data storage and centralized control by the Database Administrator (DBA) help minimize unnecessary duplication. Reducing redundancy saves storage space, decreases processing overhead, and simplifies data maintenance.

2) **Improved Data Sharing:** A DBMS enables multiple users and application programs to access and share the same data efficiently. This allows different applications to use a common database while maintaining controlled and organized access to the data.
3) **Data Integrity:** Data integrity ensures that the data stored in the database is accurate, valid, and reliable. A DBMS allows the DBA to define and enforce integrity constraints that control the values that can be stored in the database. For example, in a customer database, a constraint can be defined to allow customer records only for customers located in specific cities, such as Noida or Meerut.
4) **Data Security:** A DBMS provides mechanisms for protecting data against unauthorized access. The DBA can define authorization rules and access controls that determine which users can access particular data and what operations they are permitted to perform. This is especially important when dealing with sensitive or confidential information.
5) **Data Consistency:** Reducing data redundancy also helps maintain data consistency. When a particular data item is stored in only one location, there is less possibility of having conflicting versions of the same data. For example, if a customer's address is stored only once, updating the address requires changing it in only one location. This reduces inconsistencies and simplifies data maintenance while also avoiding unnecessary storage consumption.
6) **Efficient Data Access:** A DBMS provides efficient mechanisms for storing, retrieving, and processing data. Since applications access the data through the DBMS, the system can use various techniques, such as indexing and query optimization, to improve data retrieval and processing performance.

7) **Enforcement of Standards:** Because data is centrally managed, the DBA can establish and enforce standards throughout the database system. These standards may include naming conventions, data formats, data quality requirements, and documentation standards. Enforcing common standards improves the organization and quality of the stored data.
8) **Data Independence:** Data independence is the ability to modify the structure or representation of data without requiring major changes to application programs. In a database system, the DBMS provides an interface between application programs and the stored data. When changes are made to the physical or logical representation of the data, the DBMS can handle the necessary transformations while continuing to provide applications with data in the required form.
9) **Reduced Application Development and Maintenance Time:** A DBMS provides many functions that are commonly required by database applications, such as data storage, retrieval, security, transaction management, and concurrency control. Developers can use these built-in capabilities instead of implementing them from scratch, which can significantly reduce application development and maintenance time.

1.7 Disadvantages of a DBMS

1) **Complexity:** A DBMS provides a wide range of features and functionalities to support different users and applications. As a result, the underlying software can be complex. Database designers, developers, and administrators need appropriate knowledge and expertise to use and manage the system effectively.
2) **High Memory and Resource Requirements**: Because of its extensive functionality, a DBMS generally requires significant amounts of memory, processing power, and storage resources. Adequate hardware resources may therefore be necessary to ensure efficient system performance.

3) **Impact of System Failure:** A DBMS is often used as a centralized system that provides data access to many users and applications. Therefore, a failure of the database server or DBMS can affect a large number of users simultaneously. Appropriate backup, recovery, and fault-tolerance mechanisms are required to reduce the impact of such failures.
4) **Performance Overhead:** A DBMS is designed as general-purpose software to support a wide range of applications rather than a single specialized application. The additional features and management functions provided by the DBMS may introduce some processing overhead. Consequently, certain specialized applications may perform more slowly than they would with a dedicated data-management solution.

1.8 Benefits of Database Systems

Database systems provide several important benefits that improve the efficiency, reliability, flexibility, and maintainability of data management. The major benefits include the following:

1. **Organization of Data at the Organizational Level:** In a database environment, data is designed and organized to serve the needs of the **entire organization**, rather than being developed for a single application. The

DBMS manages the structure of data records and the relationships among them. This centralized approach reduces the amount of data-management work that application developers need to perform and promotes consistency across different applications.

2. **Extensive Data Sharing and Easy Expansion:** A database allows data to be shared among multiple applications and authorized users. This reduces unnecessary duplication of data and can save storage resources. Centralized data sharing also helps prevent incompatibility and inconsistencies that may occur when different applications maintain separate copies of the same information. Furthermore, database systems are designed by considering the overall requirements of an organization. A well-structured and flexible database can therefore be expanded more easily when new applications, users, or business requirements are introduced.
3. High Degree of Data Independence: One of the major advantages of a DBMS is data independence, which separates applications from the details of how data is stored and organized. Data independence can be considered at two levels:
    a. **Physical data independence:** The physical storage characteristics of the data are managed by the DBMS. Applications interact primarily with the logical representation of the data and generally do not need to be modified when changes are made to storage structures, file organization, or storage devices.
    b. **Logical data independence:** Applications can continue to operate with minimal or no modification when certain changes are made to the logical structure of the database. Data independence simplifies application development and reduces the complexity and maintenance

effort associated with database applications. In effect, it creates a separation between data and applications, reducing the strong dependency that commonly exists in traditional file-processing systems. This separation improves flexibility and makes systems easier to develop, modify, and maintain.

1) **Centralized Management and Control:** A DBMS provides a unified environment for managing and controlling organizational data. It supports several important functions, including data security, data integrity, concurrency control, and database recovery. Data security: Protects data against unauthorized access, disclosure, modification, or destruction. The DBMS can control which users are allowed to access particular data and what operations they are permitted to perform. Data integrity: Ensures that stored data remains accurate, valid, and consistent. Integrity constraints can be used to restrict data values to acceptable ranges and to enforce required relationships among related data. Concurrency control: Coordinates simultaneous access to the database by multiple users or applications. It ensures that concurrent operations do not interfere with one another or produce incorrect or inconsistent results.
2) **Database recovery:** Enables the DBMS to restore the database to a correct and consistent state following failures such as hardware problems, software errors, transaction failures, or incorrect user operations. Through these capabilities, a DBMS provides a controlled and reliable environment for storing, sharing, and managing organizational data.

1.9 Purpose of Database Systems

Database systems were developed in response to the limitations of early approaches to computerized data management. In the early days of computing, organizations

commonly stored their data in separate operating-system files and developed individual application programs to process those files. To illustrate this approach, consider a university that needs to maintain information about instructors, students, departments, and courses. One traditional approach is to store this information in a collection of files managed by the operating system. Application programs are then developed to perform specific operations on these files, such as:

- Adding new students, instructors, and courses.
- Registering students for courses and generating class rosters.
- Assigning grades, calculating grade point averages (GPAs), and generating student transcripts.

These application programs are developed by system programmers according to the university's requirements. As new requirements arise, additional programs and files are created. For example, suppose that a university introduces a new major, such as Computer Science. The university may create a new department and add files containing information about instructors, students, courses, and degree requirements. New application programs may also be required to implement rules specific to the new major. Similarly, whenever the university introduces new policies or requirements, additional application programs may need to be developed.

Over time, this approach can result in a large collection of files and application programs. Such an environment is known as a file-processing system. A conventional operating system supports the file-processing system by managing the files in which permanent records are stored. Different application programs are required to retrieve, insert, update, and delete records in the appropriate files. Although file-processing systems were useful in the early stages of computerized

data management, they have several significant limitations. These limitations motivated the development of Database Management Systems (DBMSs).

- **Data Redundancy and Inconsistency:** Data redundancy occurs when the same information is stored in multiple locations. In a file-processing system, different programmers may create files and application programs independently over a long period of time. Consequently, different files may have different structures, and applications may be developed using different programming languages. For example, suppose a student has a double major in Music and Mathematics. The student's address and telephone number might be stored in both the Music department's student file and the Mathematics department's student file. Storing the same information in multiple locations increases storage requirements and may also increase data-access and maintenance costs.

  More importantly, redundancy can lead to data inconsistency, where different copies of the same data contain different values. For example, if the student changes their address, the new address may be updated in the Music department's file but not in the Mathematics department's file. The database would then contain conflicting information about the same student.
- **Difficulty in Accessing Data:** File-processing systems make it difficult to retrieve information that was not anticipated when the original application programs were developed. For example, suppose a university clerk needs a list of all students who live within a particular postal-code area. If the original system does not contain an application program for generating such a list, the clerk has limited options. The clerk could either:

- Generate a list of all students and manually identify those living in the required postal-code area; or
- Ask a programmer to develop a new application program.

Neither option is efficient.

Suppose that a few days later, the clerk needs a list of students from the same postal-code area who have completed at least **60 credit hours**. A new program would again be required. This example illustrates a major limitation of conventional file-processing systems: they do not provide users with a flexible and convenient way to retrieve information according to changing requirements. Database systems address this problem by providing powerful **query languages**, such as SQL, that allow users to retrieve and manipulate data efficiently.

- **Data Isolation:** In a file-processing environment, data is often distributed across multiple files, and different files may use different formats and structures. Because the required information is isolated in separate files, developing new applications that combine and process data from multiple files can be difficult and time-consuming. A DBMS provides an integrated approach to data management, allowing applications and users to access related data without needing to know the physical organization of individual files.
- **Integrity Problems:** The data stored in a database should satisfy certain integrity constraints to ensure that it remains accurate and valid. For example, suppose a university maintains an account for each department and stores the balance of each account. The university may impose a constraint stating that a department's account balance must never fall below zero. In a traditional

file-processing system, developers typically enforce such rules by including appropriate code in individual application programs. If new integrity constraints are introduced, existing programs may need to be modified. This becomes particularly difficult when a constraint involves data stored in several different files. A DBMS provides mechanisms for defining and enforcing integrity constraints centrally, making it easier to maintain the validity and consistency of the data.

- **Atomicity Problems:** Computer systems are subject to hardware and software failures. In many applications, it is essential that a sequence of related operations be completed entirely or not performed at all. Consider a program that transfers $500 from the account of department A to the account of department B. The transfer involves two operations:
    - Deducting $500 from department A's account.
    - Adding $500 to department B's account.

  If a system failure occurs after the amount has been deducted from department A but before it is added to department B, the database will be left in an inconsistent state. Therefore, the transfer must be atomic: either both operations are completed successfully, or neither operation takes effect. Ensuring this type of atomic behavior is difficult in a conventional file-processing system. A DBMS provides transaction management and recovery mechanisms to ensure that database operations maintain consistency even when failures occur.

- **Concurrent-Access Anomalies:** Modern information systems often allow multiple users to access and update data simultaneously. Concurrent access improves system performance and allows many users to work with the

database at the same time. However, without appropriate control mechanisms, simultaneous updates can produce incorrect or inconsistent results. Consider a department account with a balance of **$10,000**. Suppose two clerks simultaneously withdraw $500 and $100, respectively.

- Ideally, the final balance should be:
- $10,000 − $500 − $100 = $9,400

However, if both application programs read the original balance of $10,000 before either update is written, one program may calculate $9,500 while the other calculates $9,900. If the two programs then write their results independently, the final balance may become either $9,500 or $9,900 instead of the correct value of $9,400. A DBMS addresses such problems using concurrency-control mechanisms, which coordinate simultaneous transactions and help ensure that concurrent operations produce correct results.

Example: Course Registration

Consider a university course with a registration limit of 40 students. Suppose the current number of registered students is 39. Two students attempt to register for the course at approximately the same time. Each registration program may:

i- Read the current number of registered students.
ii- Check whether the limit has been reached.
iii- Increase the count by one.
iv- Store the updated count.

If both programs read the value 39 before either update is completed, both may calculate and store the value 40. The database would therefore indicate that only one additional student was registered, even though two students were successfully registered. Furthermore, if the registration limit is 40, both students

may be allowed to register, resulting in 41 students in a course whose maximum capacity is 40. A DBMS prevents such problems through appropriate transaction and concurrency-control mechanisms.

- **Security Problems:** Not every user should have access to all information stored in a database. Different users should have access only to the data required for their responsibilities. In a traditional file-processing system, application programs are often developed independently and incrementally. As a result, enforcing consistent and centralized security policies can be difficult. A DBMS provides mechanisms such as user authentication, authorization, access privileges, and role-based access control to restrict access to sensitive data.

1.10 Database Architecture

Database architecture describes the main components of a database system and how they interact. It is influenced by the underlying computer architecture on which the database system operates. Database systems shown in Figure 1.5 can be implemented using different architectures, including:

1. **Centralized systems:** The database and its processing are managed by a single computer.
2. **Client–Server systems:** A server provides database services to multiple client machines.
3. **Parallel systems:** Database operations are distributed across multiple processors to improve performance.
4. **Distributed databases:** Data is stored and managed across multiple geographically separated computers.

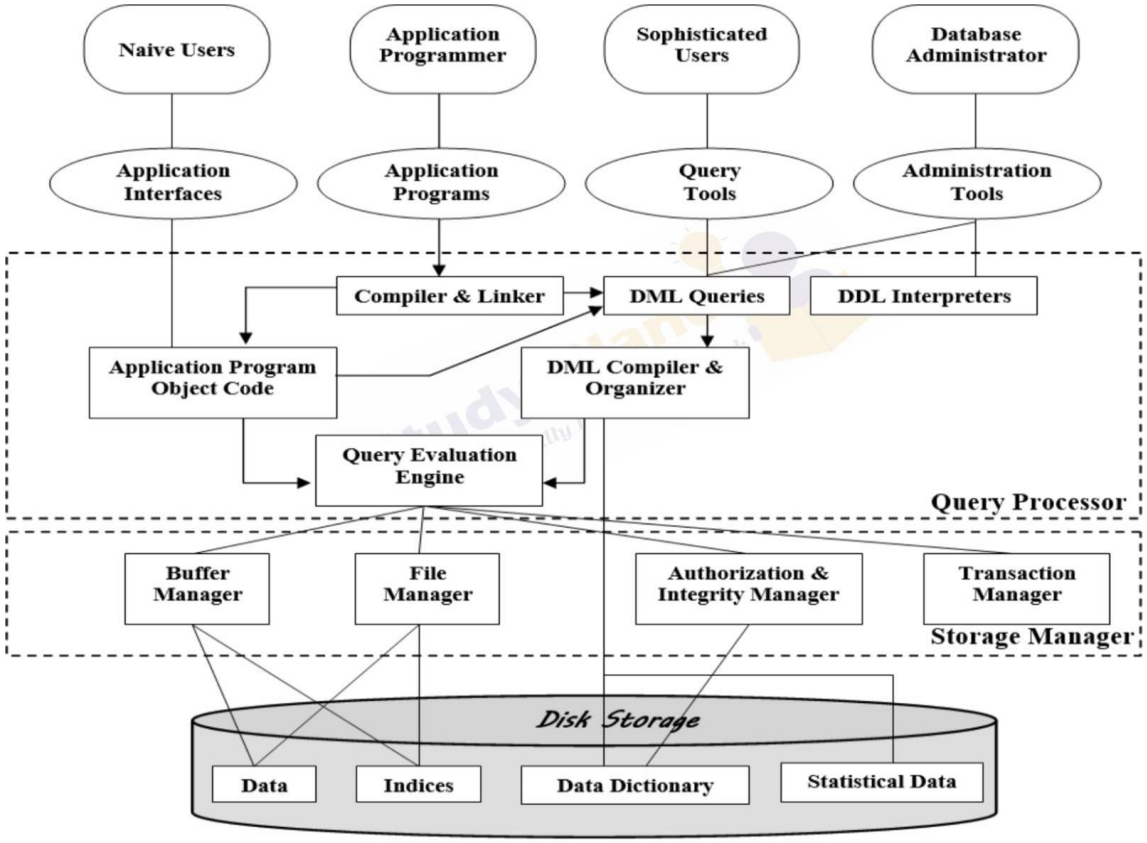


Figure 1.5 Database System

1.10.1 Main Components of a Database System

A database system is divided into several modules, with two major functional components:

1. **Storage Manager:** Manages the storage, retrieval, and organization of data on physical storage devices. It is essential because databases often contain large amounts of data.
2. **Query Processor:** Processes queries and updates submitted by users and translates them from high-level, nonprocedural commands into efficient operations that can be executed at the physical level.

Thus, the database system acts as a bridge between the **logical level**, where users specify what data they need, and the **physical level**, where the required operations are actually performed.

1.10.2 Database Application Architectures

Database applications are commonly organized using **two-tier** or **three-tier** architectures as decribed in Figure 1.4.

1. **Two-Tier Architecture:** In this architecture, the application runs on the client machine and communicates directly with the database server using database queries. The client application sends requests to the database server, which processes the requests and returns the results. Standards such as ODBC and JDBC can be used to establish communication between the application and the database.
2. **Three-Tier Architecture:** In a three-tier architecture, the client acts mainly as a front end and does not directly communicate with the database. The three layers are:

1. **Client Layer:** Provides the user interface.
2. **Application Server:** Contains the application's business logic and processes client requests.
3. **Database Server:** Stores and manages the database.

The application server communicates with the database system to retrieve or modify data. This architecture centralizes business logic, improves maintainability and security, and is particularly suitable for large-scale and web-based applications.

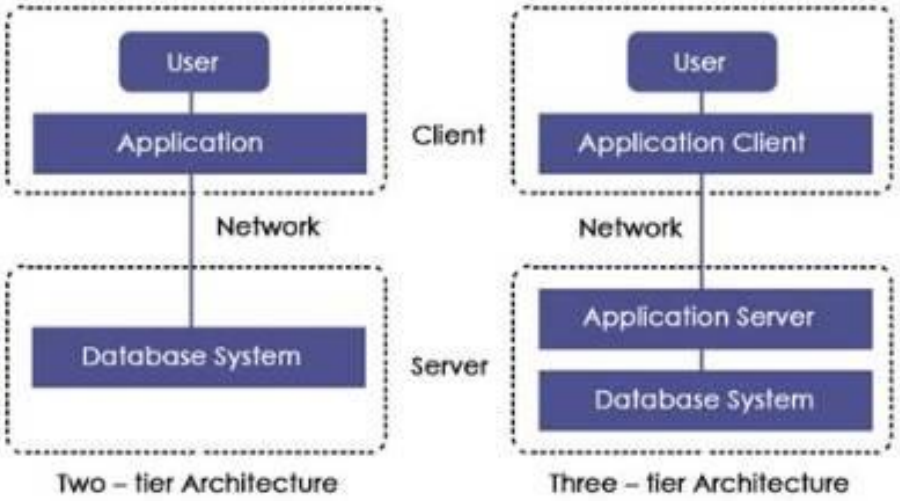


Figure 1.6 Two-tier and three-tier architectures.

1.11 Why Database Systems Were Developed

The limitations of traditional file-processing systems—such as data redundancy, data inconsistency, difficulty in accessing data, data isolation, integrity problems, atomicity problems, concurrent-access anomalies, and security issues—created the need for a more efficient and reliable approach to data management. Database Management Systems (DBMSs) were developed to address these problems. A DBMS provides centralized mechanisms for storing, retrieving, updating, securing, and managing data while supporting multiple users and applications.

1.12 View of Data

A database system consists of a collection of interrelated data and a set of programs that enable users to access, retrieve, and modify that data. One of the primary purposes of a database system is to provide users with an abstract view of the data. This means that the database system hides the complex details of how data is physically stored, organized, and maintained. Users can interact with the data through a simplified representation without needing to understand the

underlying storage mechanisms or implementation details. This separation between the user's view of the data and its physical storage makes database systems easier to use, manage, and maintain.

1.13 Data Abstraction

Achieving high performance often requires the use of complex data structures and storage techniques to represent and manage data internally. However, most database users do not need to understand these implementation details. To simplify interaction with the database, a DBMS hides the underlying complexity and presents users with a simpler representation of the data. This concept is known as data abstraction. It allows users to work with data without needing to know how the data is physically stored, organized, or processed. A database system generally provides three levels of data abstraction:

1. **Physical level (or Internal View / Schema):** The lowest level of abstraction describes how the data are actually stored. The physical level describes complex low-level data structures in detail.
2. **Logical level (or Conceptual View / Schema):** The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

3. **View level (or External View / Schema):** The highest level of abstraction describes only part of the entire database. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database. Figure 1.7 shows the relationship among the three levels of abstraction.

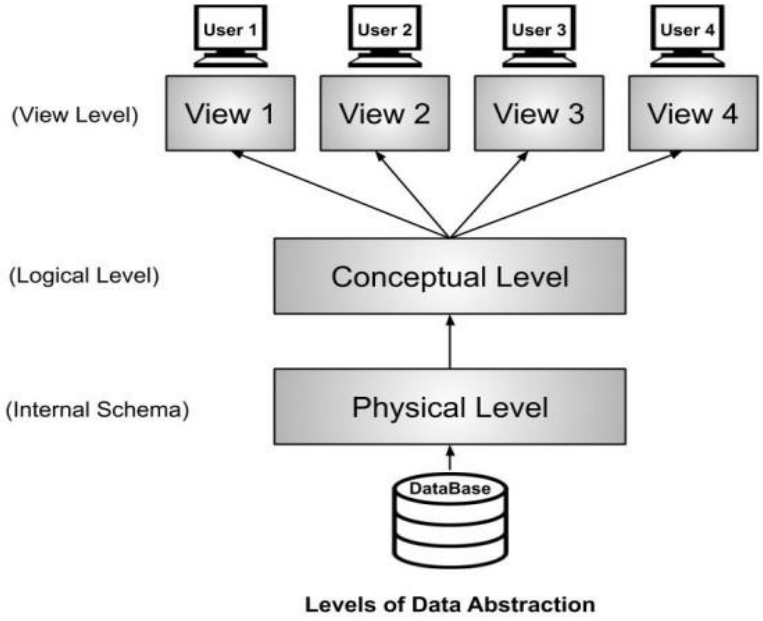


Figure 1.7 Database System

1.14 Instances and Schemas

A **database instance** is the collection of data stored in a database at a specific point in time. The data changes whenever records are inserted, updated, or deleted. A **database schema** is the overall structure or design of the database. Unlike an instance, the schema changes infrequently. A database system can have different schemas at different abstraction levels:

- **Physical schema:** Describes how data is physically stored.
- **Logical schema:** Describes the structure of the data and relationships among data items.
- **View schema:** Describes the specific parts of the database visible to particular users.

The logical schema is the most important for application development. **Physical data independence** means that changes to the physical storage of data do not require changes to application programs.

1.15 Data Models

A **data model** is a collection of concepts used to describe the structure of a database, including its data, relationships, semantics, and constraints. It provides a way to represent a database at different levels of abstraction.The main types of data models are:

1.15.1 Relational Model

The **relational model** represents data and relationships using **tables (relations)**. Each table consists of rows and columns, where columns represent attributes. It is the most widely used data model in modern database systems.

1.15.2 Entity-Relationship (E-R) Model

The **E-R model** represents a database using **entities** and the **relationships** among them. It is widely used during database design.

1.15.3 Object-Based Data Model

The **object-based model** represents data as objects that combine data with methods. It supports concepts such as **encapsulation, methods, and object identity**. The

object-relational model combines the features of object-oriented and relational models.

1.15.4 Semi-Structured Data Model

The semi-structured model allows data items of the same type to have different attributes. XML (Extensible Markup Language) is a common example of a format used to represent semi-structured data. The hierarchical and network models were developed before the relational model. They are less commonly used today and are mainly found in some older database systems.

1.16 Database Languages

A database system provides languages for defining and manipulating data:

Data-Definition Language (DDL): Used to define the database schema and its structure.

Data-Manipulation Language (DML): Used to retrieve, insert, update, and delete data.

In practice, DDL and DML are usually parts of a single database language, such as **SQL (Structured Query Language)**.

1.16.1 Data-Definition Language (DDL)

It is used to access and manipulate data stored in a database. The main operations are:

- **Retrieval:** Obtaining information from the database.
- **Insertion:** Adding new information.
- **Deletion:** Removing existing information.
- **Modification:** Updating stored information.

There are two main types of DML:

- **Procedural DML:** Specifies what data is needed and how to retrieve it.
- **Declarative (Nonprocedural) DML:** Specifies what data is needed without specifying how to retrieve it.

Declarative DMLs are generally easier to use because the DBMS determines the most efficient way to access the required data. A **query** is a statement used to request information from a database. The part of a DML used for retrieving information is called a **query language**. In practice, the terms *query language* and *DML* are often used interchangeably.

1.16.2 Data-Definition Language (DDL)

It is used to define the database schema and specify additional properties of the data. It can also define storage structures and access methods used by the DBMS. The DDL can be used to specify integrity constraints that ensure the data remains accurate and consistent. There are three types of integrity constraints listed bellows:

- **Domain Constraints:** Define the valid values for an attribute, such as integer, character, or date values.
- **Referential Integrity:** Ensures that a value in one table correctly references an existing value in another table. For example, every department specified for a course must exist in the Department table.
- **Assertions:** Define general conditions that must always be satisfied by the database. For example, every department may be required to offer at least five courses each semester.
- **Authorization:** Controls what operations users are allowed to perform. Common permissions include **read, insert, update, and delete**.

1.17 Data Dictionary

The output of DDL statements is stored in the **data dictionary**, which contains **metadata**, or information about the database structure and its data. The data dictionary stores information such as table definitions, attributes, data types, constraints, and user privileges. The DBMS uses this information when accessing and modifying the database. A **data dictionary** is a DBMS component that stores descriptions of data, their characteristics, and relationships. This information is known as **metadata**, or "data about data." It provides the DBMS with a self-describing capability and supports database administration and management. Data dictionaries can be classified into two main types:

1. **Integrated Data Dictionary:** Built into the DBMS and automatically maintained by the system. Relational DBMSs commonly provide a built-in data dictionary, also called a **system catalog**.
2. **Stand-Alone Data Dictionary:** Operates separately from the DBMS and can be used by database administrators to document and manage data from different systems. They can also be classified as:
    - **Active Data Dictionary:** Automatically updated by the DBMS whenever database changes occur. Thus, database systems provide different interfaces and tools to support users with different levels of technical expertise and different database requirements.
    - **Passive Data Dictionary:** Not automatically updated and usually requires a separate process to maintain its information.

1.17.1 Functions of a Data Dictionary

The main function of a data dictionary is to store information about the objects that interact with the database. It improves communication between database designers, administrators, developers, and end users. It can also help resolve data conflicts and support query optimization. A typical data dictionary may contain information about:

- **Data elements:** Names, data types, formats, storage details, validation rules, and where each element is used.
- **Tables:** Table names, creators, creation dates, number of columns, and access permissions.
- **Indexes:** Index names, indexed attributes, locations, characteristics, and creation dates.
- **Databases:** Database names, creators, locations, creation dates, and administrators.
- **Users and administrators:** Information about database users and administrators.
- **Applications and programs:** Programs that access the database, including forms, reports, and SQL queries.
- **Access privileges:** Authorization and permissions assigned to users.
- **Relationships:** Relationships between data elements, including their constraints, connectivity, and cardinality.

Thus, the data dictionary is an essential tool for **documenting, managing, and controlling database metadata**.

1.18 Database Administrators and Database Users

A primary objective of a database system is to store, retrieve, and manage information efficiently. People who interact with a database are generally classified as database users or database administrators (DBAs).

1.18.1 Database Users and User Interfaces

Database users can be classified into four main types according to how they interact with the database system. Different user interfaces and tools are provided to support their needs.

1. Naive Users: Naive users interact with the database through predefined application programs rather than directly issuing database commands. For example, a bank teller may use a transfer application to transfer money from one account to another. The application asks for the required information and performs the transaction automatically. Another example is a customer checking an account balance through a website by entering an account number. The application retrieves the required information from the database and displays it to the user. The most common interface for naive users is a forms-based interface. They may also receive information through predefined reports.

2. Application Programmers: Application programmers are professionals who develop application programs that access and manipulate databases. They use programming languages, database APIs, and development tools to create user interfaces and database applications. Rapid Application Development (RAD) tools can help programmers create forms and reports with minimal programming. Some database systems also provide fourth-generation languages (4GLs) that combine programming constructs with database manipulation commands.

3. Sophisticated Users: Sophisticated users interact directly with the database without developing complete application programs. They use query languages, such as SQL, to retrieve and analyze data. Their queries are processed by the query processor, which translates database commands into instructions that can be executed by the database system. Examples include data analysts and managers who need to explore and analyze organizational data. Online Analytical Processing (OLAP) tools help users examine summarized data from different perspectives. For example, sales can be analyzed by region, product, or a combination of both. Users can also drill down to more detailed information or aggregate data into higher-level categories. Data mining tools are another important category. They help users discover patterns, trends, and relationships within large datasets.

4. Specialized Users: Specialized users are advanced users who develop or use specialized database applications that do not follow traditional data-processing methods. Examples include:

- Computer-aided design (CAD) systems.
- Knowledge-based and expert systems.
- Applications that manage complex data such as graphics, audio, and video.
- Environmental and scientific modeling systems.

1.19 Query Processor

The **query processor** is responsible for interpreting, compiling, optimizing, and executing database queries and commands. Its main components are:

- **DDL Interpreter:** Processes DDL statements and records the database definitions in the **data dictionary**.

- **DML Compiler:** Translates DML statements into an **evaluation plan** consisting of low-level instructions. It also performs **query optimization** by selecting the most efficient plan among alternative plans that produce the same result.
- **Query Evaluation Engine:** Executes the low-level instructions generated by the DML compiler and produces the required results.

1.19.1 Storage Manager

The storage manager provides an interface between the database's physical data and the application programs and queries. It communicates with the file system and translates database operations into low-level file-system operations. The storage manager is responsible for storing, retrieving, and updating data in the database. Its main components are:

- **Authorization and Integrity Manager:** Verifies user access privileges and ensures that database integrity constraints are satisfied.
- **Transaction Manager:** Maintains database consistency during transaction execution and ensures correct operation in the event of system failures or concurrent transactions.
- **File Manager:** Manages disk space allocation and the data structures used to organize data on storage devices.
- **Buffer Manager:** Transfers data between disk storage and main memory and manages the data cached in memory. It allows the DBMS to efficiently handle databases that are much larger than available main memory.

1.19.2 Transaction Manager

A transaction is a sequence of database operations that performs a single logical task. It is treated as a single unit to ensure atomicity and consistency. A transaction must preserve the database's consistency constraints. Therefore, if the database is in a consistent state before a transaction begins, it should remain consistent after the transaction completes successfully. The Transaction Manager is responsible for maintaining database consistency despite system failures, such as power outages and operating system crashes, as well as transaction failures. It also coordinates concurrent transactions to ensure correct and reliable database operations.

Sheet1:

1. What is the primary purpose of a database system?

A. To replace the operating system

B. To store, manage, and process data efficiently

C. To design computer hardware

D. To create programming languages

2. Which of the following can be considered data?

A. Numbers only

B. Text only

C. Images only

D. Numbers, text, images, audio, and video

3. What does the term "semantics" of data refer to?

A. The physical storage location

B. The meaning or significance of the data

C. The size of the data

D. The security level of the data

4. Which of the following is a characteristic of a database?

A. Data is stored only temporarily

B. Data cannot be shared

C. Data is stored for long-term use

D. Data can only be accessed by one user

5. What is a DBMS?

A. A programming language

B. Software used to manage databases

C. A computer network

D. An operating system

6. Which language is used to define the structure of a database?

A. DML

B. DDL

C. HTML

D. XML

7. Which of the following is a function of DML?

A. Defining the database schema

B. Retrieving and modifying data

C. Designing computer networks

D. Managing computer hardware

8. Which of the following is an advantage of a DBMS?

A. Increased data redundancy

B. Reduced data sharing

C. Improved data security

D. Increased data inconsistency

9. What is data redundancy?

A. Protecting data from unauthorized access

B. Unnecessary duplication of data

C. Removing all data from a database

D. Encrypting database records

10. Which DBMS feature protects data from unauthorized users?

A. Data security

B. Data redundancy

C. Data isolation

D. Data duplication

11. Which of the following is a disadvantage of a DBMS?

A. Improved data sharing

B. Data security

C. System complexity

D. Data independence

12. What does physical data independence mean?

A. Users cannot access physical data

B. Physical storage changes can be made without changing applications

C. Data cannot be modified

D. Data is stored without a schema

13. Which of the following is a major limitation of traditional file-processing systems?

A. Data redundancy and inconsistency

B. Data independence

C. Centralized management

D. Improved security

14. What is data isolation?

A. Storing all data in one file

B. Data being stored in separate files with different formats or structures

C. Encrypting all database records

D. Removing duplicate data

15. What does atomicity mean in a database transaction?

A. Only the first operation must be completed

B. All operations are completed or none are completed

C. Transactions can never fail

D. Only one user can access the database

16. Which problem occurs when multiple users access and modify data at the same time?

A. Data abstraction

B. Concurrent-access problem

C. Data definition

D. Data independence

17. Which database architecture uses a single computer system?

A. Distributed architecture

B. Parallel architecture

C. Centralized architecture

D. Client-server architecture

18. In a client-server architecture, what does the server provide?

A. Database services

B. Keyboard input

C. Operating system installation

D. Programming lessons

19. Which of the following is a major component of a DBMS?

A. Storage manager

B. Web browser

C. Compiler only

D. Text editor

20. What is the main role of a query processor?

A. To design hardware

B. To interpret, optimize, and execute database queries

C. To manage computer networks

D. To create user accounts only

21. Which component interprets DDL statements?

A. Buffer manager

B. DDL interpreter

C. File manager

D. Transaction manager

22. What is the purpose of query optimization?

A. To increase data redundancy

B. To find an efficient way to execute a query

C. To delete database schemas

D. To prevent all users from accessing data

23. Which component manages files and data structures stored on disk?

A. File manager

B. Query optimizer

C. DDL interpreter

D. Application programmer

24. What is the main function of the buffer manager?

A. Managing users' passwords

B. Transferring data between disk storage and main memory

C. Creating database schemas

D. Writing application programs

25. Which level of data abstraction describes how data is physically stored?

A. View level

B. Logical level

C. Physical level

D. External user level

26. What is a database schema?

A. The data stored at a particular moment

B. The overall logical structure of the database

C. A database transaction

D. A database user

27. Which data model represents data using tables?

A. Relational model

B. E-R model

C. Object-based model

D. Semi-structured model

28. What is a data dictionary?

A. A programming language

B. A collection of information about the data in a database

C. A backup copy of a database

D. A database application

29. Which type of database user typically interacts with predefined forms?

A. Sophisticated user

B. Specialized user

C. Naive user

D. Database administrator

30. What is a transaction?

A. A database table

B. A sequence of operations that performs a single logical task

C. A type of database schema

D. A database user

Chapter 2: Entity Relationship Diagram (ERD)

2.1 Introduction

Designing a database is an essential step in developing reliable and efficient information systems. A well-designed database should organize data in a clear and meaningful way, minimize unnecessary duplication, maintain data consistency, and support the requirements of its users and applications. Before creating tables and implementing a database using a specific Database Management System (DBMS), it is important to understand the data requirements and represent the relationships among the different types of data.

Entity Relationship (ER) Modeling is one of the most widely used techniques for conceptual database design. It provides a graphical and easy-to-understand representation of the data requirements of an organization or system. ER modeling focuses on identifying the important entities, their attributes, and the relationships

that exist among them, without being concerned with the details of a particular database implementation. The ER model provides a common language between database designers, developers, and users. By representing the real-world environment using entities and relationships, it helps designers analyse the system requirements before converting them into relational database tables. This approach can also reveal missing information, unnecessary data, and inconsistencies at an early stage of the database development process.

This chapter introduces the fundamental concepts of ER modeling and explains how they are used to develop a conceptual database design. It covers entities, attributes, relationships, keys, cardinality, participation constraints, weak entities, and other important ER-modeling concepts. It also explains how to construct and interpret Entity Relationship Diagrams (ERDs) and how ER models can be transformed into relational database schemas as shown in Figure 2.1. By the end of this chapter, students should be able to identify entities and their attributes, determine relationships between entities, specify appropriate constraints, construct ER diagrams, and use ER modeling as a foundation for designing well-structured relational databases.

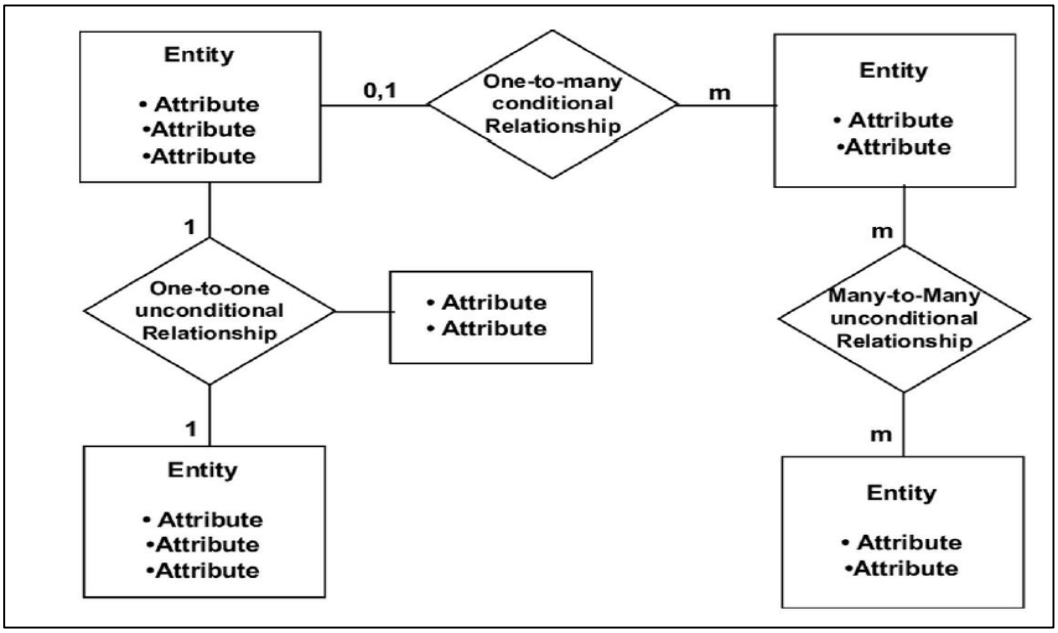


Figure 2.1 Entity Relationship Diagrams (ERDs)

2.2 Database Design Techniques

There are two common techniques are used for database design:

1. **ERD (Top-Down Approach):** It is a graphical technique used to represent and organize data independently of the actual database implementation. It describes entities, their attributes, and the relationships among them. ERDs are commonly used during the database design phase to visualize and communicate the structure of a database system. It Begins with high-level concepts such as entities and relationships and gradually develops the database structure.
2. **Normalization (Bottom-Up Approach):** Starts with existing data requirements and organizes them into well-structured relations to reduce redundancy and dependency problems.

2.2.1 Key Components of an ERD

1. Entity

An **entity** is an object or concept that has an independent existence and about which data is collected. Examples include Employee, Student, and Department. Entity may be instance, Strong, and Weak.

a. Entity Instance

It is a specific occurrence or member of an entity type. For example, a particular employee is an instance of the **Employee** entity.

b. Strong (Regular) Entity

A **strong entity** has its own key attribute that uniquely identifies each instance. For example, **Employee** can be a strong entity if each employee has a unique Employee_ID.



c. Weak Entity

A **weak entity** depends on another entity for its existence and does not have a complete key of its own. The related independent entity is called the **strong entity**. For example, a **Child** entity may depend on a **Parent** entity.

2. Attributes



An **attribute** is a property or characteristic that describes an entity. For example, Employee may have attributes such as Employee_ID, Name, and Salary.



The **domain** of an attribute is the set of all valid values that the attribute can take. For example, the domain of a Day attribute may contain values such as Monday, Tuesday, and so on.

i- Key Attribute

A **key attribute** is an attribute, or a combination of attributes, that uniquely identifies each entity instance.

ii- Simple Attribute

A **simple attribute** cannot be divided into smaller meaningful components. For example, Employee_ID can be considered a simple attribute.

iii- Composite Attribute

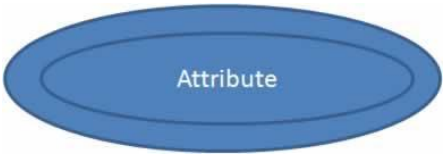
A **composite attribute** can be divided into smaller components. For example, Name can be divided into First_Name, Middle_Name, and Last_Name.

iv- Single-Valued Attribute

A **single-valued attribute** has only one value for each entity instance. For example, a student's age normally has one value at a particular time.

v- Multivalued Attribute

A **multivalued attribute** can have more than one value for an entity instance. For example, an employee may have multiple telephone numbers.



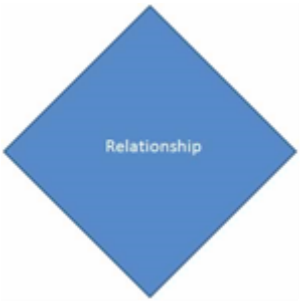
ii- **Stored Attribute:** It is a value that is stored directly in the database. For example, Date_of_Birth is a stored attribute.

iii- **Derived Attribute:** It is calculated from one or more other attributes. For example, Age can be derived from Date_of_Birth and the current date.



3. Relationships

A **relationship** represents an association between two or more entities. For example, the statement "An employee works for a department" represents a relationship between Employee and Department. A relationship involving a weak entity is represented using an appropriate identifying relationship as given below.



The **degree of a relationship** is the number of entity types participating in the relationship. However in ER Modeling, To connect a weak Entity with

others, you should use a weak relationship notation as given below:



- **Unary relationship:** Involves one entity type. Example: An employee manages another employee.
- **Binary relationship:** Involves two entity types. Example: An employee works for a department.
- **Ternary relationship:** Involves three entity types. Example: A customer purchases an item from a supplier. An **n-ary relationship** involves n entity types.

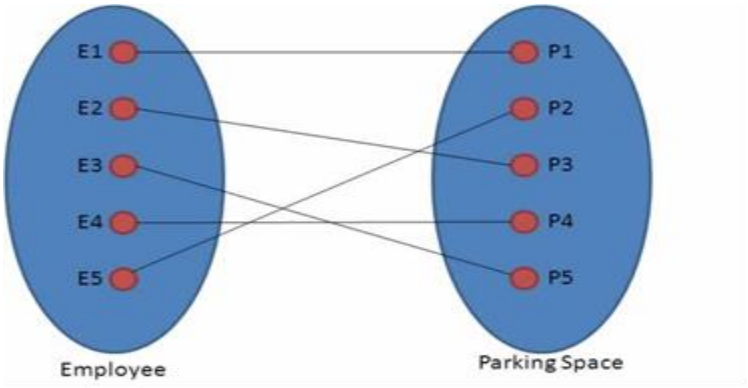
4. Cardinality of a Relationship

It specifies how many instances of one entity can be associated with instances of another entity.The main types are:

1. **One-to-One (1:1):** Each entity instance is associated with at most one instance of the other entity. Example: An employee is assigned to one parking space.
2. **One-to-Many (1:N):** One instance of an entity can be associated with many instances of another entity. Example: One organization can have many employees.

3. **Many-to-One (N:1):** Many instances of one entity are associated with one instance of another entity. Example: Many employees work for one organization.
4. **Many-to-Many (M:N):** Many instances of one entity can be associated with many instances of another entity. Example: Students enroll in many courses, and each course can have many students.

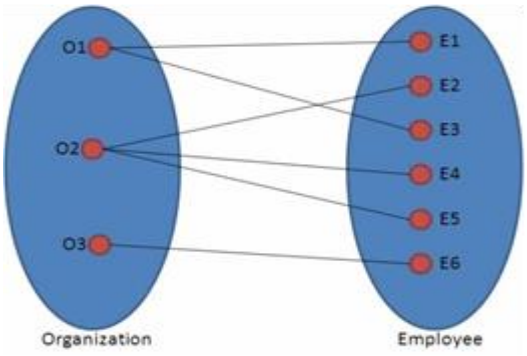
The minimum and maximum values of this connectivity is called the cardinality of the relationship. Example for Cardinality – One-to-One (1:1) Employee is assigned with a parking space.



One employee is assigned with only one parking space and one parking space is assigned to only one employee. Hence it is a 1:1 relationship and cardinality is One-To-One (1:1) In ER modeling, this can be mentioned using notations as given below:



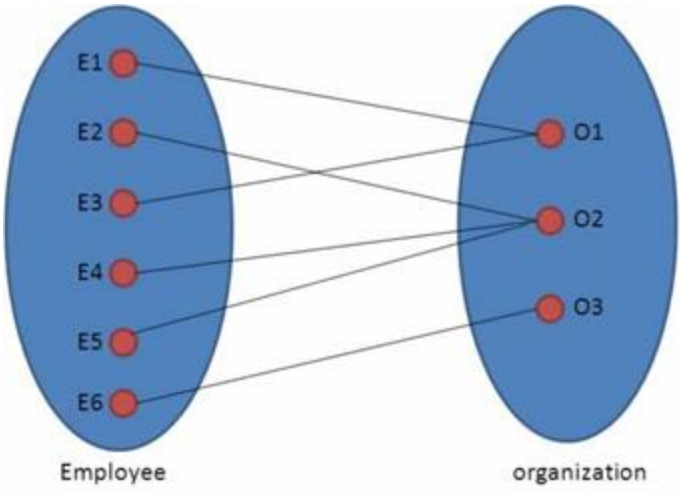
Example for Cardinality – One-to-Many (1:N) Organization has employees



One organization can have many employees, but one employee works in only one organization. Hence it is a 1:N relationship and cardinality is One-To-Many (1:N). In ER modeling, this can be mentioned using notations as given below:



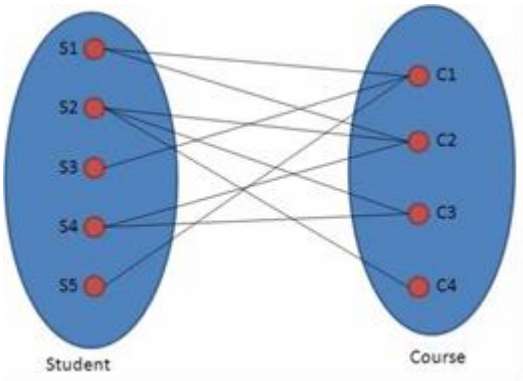
Example for Cardinality – Many-to-One (M :1): It is the reverse of the One to Many relationships. employee works in organization.



One employee works in only one organization But one organization can have many employees. Hence it is a M:1 relationship and cardinality is Many-to-One (M :1). In ER modeling, this can be mentioned using notations as given below:



Cardinality – Many-to-Many (M:N) Students enrolls for courses



One student can enroll for many courses and one course can be enrolled by many students. Hence it is a M:N relationship and cardinality is Many-to-Many (M:N). In ER modeling, this can be mentioned using notations as given below:



5. Relationship Participation

Relationship participation specifies whether all or only some entity instances participate in a relationship.

- **Total Participation:** Every instance of the entity must participate in the relationship.
- **Partial Participation:** Only some instances of the entity participate in the relationship. For example, in the relationship **Employee is Head of Department**, not every employee is a department head, so Employee has **partial participation**. However, if every department must have a head, Department has **total participation**.

2.3 Advantages of ER Modeling

- Provides a simple and easily understandable representation of a database.
- Can be understood by both technical and non-technical users.
- Helps identify entities, attributes, and relationships during database design.
- Provides a high-level description of the system.
- Supports the development of the physical database structure.

2.4 Disadvantages of ER Modeling

- Converting an ER model into a physical database may involve ambiguities or inconsistencies.
- Complex ER diagrams may sometimes be difficult to interpret.

2.5 Relational Model

The relational model is the most widely used data model for commercial database applications. Its popularity is mainly due to its simplicity, which makes database design and programming easier than earlier models, such as the hierarchical and network models.

2.5.1 Structure of Relational Databases

A relational database consists of a collection of "tables", where each table has a unique name. In the relational model, a table is called a "relation". For example, as shown in Table 2.1 the "instructor" relation stores information about instructors. It contains the attributes "ID", "name", "dept name", and "salary". Each row represents an instructor and contains values for these attributes. Similarly, as shown in Table 2.2 the "course" relation stores information about courses using attributes such as "course id", "title", "dept name", and "credits".

Table 2.1 The instructor relation

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Table 2.2 The Course relation

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

The”prereq” relation shown in Table 2.3 stores prerequisite relationships between courses. It contains two attributes: “course id” and “prereq id”. Each row indicates that one course is a prerequisite for another. In general, a row in a relational table represents a relationship among a set of values. In mathematical terminology, a row is called a “tuple”, which is an ordered sequence of values. Therefore, a table corresponds to a mathematical “relation”, which is the basis for the name “relational model”.

Table 2.3 The prereq relation

course_id	prereq_id
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

The main terminology of the relational model is:

Relation: A table in the database.

Tuple: A row in a relation.

Attribute: A column in a relation.

Relation instance: The collection of tuples stored in a relation at a particular point in time.

Domain: The set of permitted values for an attribute.

For example, the "instructor" relation has four attributes: "ID, name, dept name," and "salary". If the relation currently contains information about 12 instructors, its current instance contains 12 tuples. The order of tuples in a relation is irrelevant because a relation is considered a "set of tuples". Therefore, two tables containing exactly the same tuples represent the same relation, regardless of the order in which the tuples are displayed.

2.6 Domains and Atomic Values

Each attribute has a “domain”, which defines the set of valid values that can be stored in that attribute. For example, the domain of the “salary” attribute consists of valid salary values, while the domain of “name” consists of valid instructor names. In the relational model, attribute domains are required to be “atomic”. A domain is atomic when each value is treated as a single, indivisible unit. For example, if the “phone number” attribute stores a set of several phone numbers, its domain is not atomic because each value contains multiple phone numbers. However, if each phone number is stored and treated as a single value, the domain is considered atomic. The important consideration is not only the structure of the value but also how the value is treated by the database. A phone number may contain a country code, area code, and local number, but if it is treated as one indivisible value, the attribute has an atomic domain.

2.7 Null Values

A **null value** is a special value used to indicate that a value is unknown, unavailable, or does not exist. For example, an instructor may not have a telephone number, or the number may be unavailable. In such cases, a null value may be stored for the *phone number* attribute. Null values can introduce difficulties when querying and updating databases. Therefore, they should be avoided whenever possible.

2.8 Database Schema

It represents the logical design or structure of a database, whereas a “database instance “represents the actual data stored in the database at a particular point

in time. “relation schema” consists of the relation name, its attributes, and the corresponding domains. For example as shown in Table 2.4

department (**dept name, building, budget**)

The relation schema describes the structure of the relation, while the relation instance contains the actual tuples currently stored in it. The distinction is similar to the difference between a variable's **type** and it’s **current value** in a programming language. The value of a variable can change during program execution, while its type generally remains unchanged. Similarly, the tuples in a relation may change as data is inserted, deleted, or updated, whereas the relation schema usually remains unchanged.

Table 2.4 The prereq relation

dept_name	*building*	*budget*
Biology	Watson	90000
Comp. Sci.	Taylor	100000
Elec. Eng.	Taylor	85000
Finance	Painter	120000
History	Painter	50000
Music	Packard	80000
Physics	Watson	70000

Common attributes in different relation schemas can be used to establish relationships between relations. For example, the attribute **dept name** appears in both the *instructor* and *department* relations. This common attribute allows information from the two relations to be associated. For example, to find instructors working in the Watson building, the *department*

relation can first be examined to identify departments located in Watson. The corresponding *dept name* values can then be used to find the instructors belonging to those departments. In a university database, additional relations may be required.

For example: the relation shown in Table 2.5 shows a sample instance of the section relation. “section” (course id, sec id, semester, year, building, room number, time slot id)

Table 2.5 The Section relation

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

This relation describes individual offerings of courses. Another relation called “Teaches” shown in Table 2.6 can represent the association between instructors and course sections: ”Teaches” (**ID, course id, sec id, semester, year**)

Table 2.6 The Teaches relation

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Other commonly used relations include:

student(ID, name, dept name, tot cred)

advisor (s_id, i_id)

takes (ID, course id, sec id, semester, year, grade)

classroom (building, room number, capacity)

time slot (time slot id, day, start time, end time)

2.9 Keys

We must have a way to specify how tuples within a given relation are distinguished. This is expressed in terms of their attributes. That is, the values of the attribute values of a tuple must be such that they can uniquely identify the tuple. In other words, no two tuples in a relation are allowed to have exactly the same value for all attributes. A database must provide a way to uniquely identify each tuple within a relation. This is achieved using keys.

1. Superkey

A superkey is a set of one or more attributes that uniquely identifies each tuple in a relation. For example, in the *instructor* relation, *ID* can uniquely identify each instructor. Therefore, *ID* is a superkey. In contrast, *name* is not necessarily a superkey because two instructors may have the same name.A superkey may contain unnecessary attributes. For example, *{ID, name}* is also a superkey because *ID* alone is sufficient to identify a tuple.

2. Candidate Key

It is a minimal superkey, meaning that none of its attributes can be removed without losing its ability to uniquely identify tuples. A relation may have more than one candidate key. For example, if both *ID* and the combination *{name, dept name}* can uniquely identify instructors, both are candidate keys.

3. Primary Key

It is the candidate key selected by the database designer as the main means of uniquely identifying tuples in a relation. Primary keys should be chosen carefully. They should preferably contain attributes whose values are stable and rarely change. For example, a person's name or address may change or may not be unique, so these attributes are generally poor choices for primary keys. Instead, a unique identifier is usually preferred. In a relation schema, primary-key attributes are commonly listed first and are *underlined*.

4. **Foreign Key**
5. It is an attribute or set of attributes in one relation that refers to the primary key of another relation. For example, the *dept name* attribute in the *instructor* relation can be a foreign key referencing the dept *name* primary key of the

department relation. The relation containing the foreign key is called the *referencing relation*, while the relation containing the referenced primary key is called the *referenced relation.* A foreign key ensures that a value appearing in the referencing relation corresponds to a valid value in the referenced relation.

2.10 Referential Integrity

A referential integrity constraint ensures that values in a foreign key correspond to values in the referenced relation. For example, if an instructor belongs to a particular department, that department must exist in the *department* relation. Therefore, every value of *dept name* in the *instructor* relation must match a corresponding *dept name* value in the *department* relation. Another example involves the *section* and *teaches* relations. A section may be taught by one or more instructors. The attributes *course id, sec id, semester, and year* identify a particular section, but they do not uniquely identify a tuple in *teaches* because multiple instructors may teach the same section. Therefore, a foreign key can be defined from *teaches* to *section*, ensuring that every teaching assignment refers to an existing section.

2.11 Schema Diagrams

A *schema diagram* provides a graphical representation of the structure of a database. It shows the relations, their attributes, primary keys, and foreign-key dependencies. A database schema, along with primary key and foreign key dependencies, can be depicted by schema diagrams. Figure 2. 2 shows the schema diagram for the university organization. In a schema diagram:

* Each relation is represented by a *box*.

* The relation name appears at the top of the box.

* The attributes are listed inside the box.

* *Primary-key attributes* are underlined.

* *Foreign-key dependencies* are represented by arrows from the foreign key in the referencing relation to the corresponding primary key in the referenced relation.

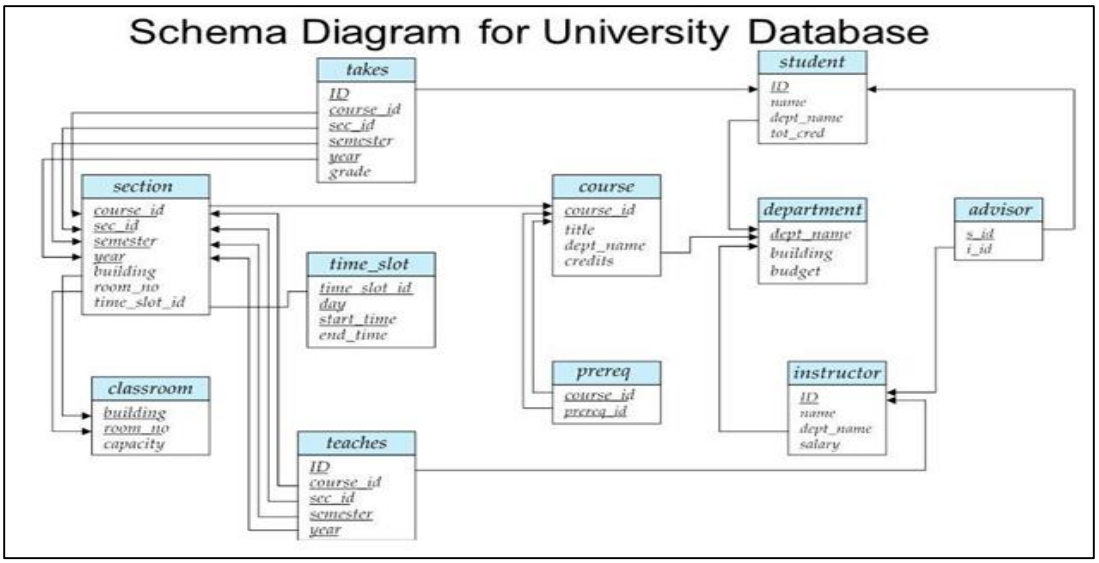


Figure 2.2 : Schema diagram for the university database.

Schema diagrams provide a convenient way to understand the structure and relationships of a relational database. However, not all referential integrity constraints can be represented using foreign-key arrows. Other diagrammatic techniques, such as *Entity-Relationship (ER) diagrams*, can be used to represent more complex relationships and constraints.

Sheet 2

1. What is the main reason for the widespread use of the relational model?

A. It requires no storage space
B. Its simplicity
C. It eliminates all database constraints
D. It does not require a DBMS

2. In the relational model, a table is called a:

A. Tuple
B. Domain
C. Relation
D. Attribute

3. A row in a relational table is called a:

A. Relation
B. Attribute
C. Tuple
D. Domain

4. A column in a relational table is called a:

A. Tuple
B. Attribute
C. Relation
D. Instance

5. Which of the following represents a collection of permitted values for an attribute?

A. Tuple
B. Domain
C. Relation
D. Schema

6. If the instructor relation has the attributes ID, name, dept name, and salary, how many attributes does it have?

A. 2
B. 3
C. 4
D. 5

7. What does a relation instance represent?

A. The logical structure of a database
B. The actual set of tuples at a particular point in time
C. The data type of an attribute
D. A database constraint

8. What is the significance of tuple order in a relation?

A. It determines the primary key
B. It determines the foreign key
C. It is important for identifying tuples
D. It is irrelevant

9. A domain is considered atomic when:

A. It contains multiple values
B. Its values are treated as indivisible units
C. It contains only numerical values
D. It contains only character values

10. Which of the following is an example of a non-atomic domain?

A. A single salary value
B. A single instructor name
C. A set of phone numbers stored as one value
D. A single course ID

11. What does a null value generally indicate?

A. A primary key
B. A duplicate value
C. An unknown, unavailable, or nonexistent value
D. A foreign key

12. Why should null values be avoided whenever possible?

A. They increase the number of attributes
B. They can cause difficulties in querying and updating data
C. They automatically delete tuples
D. They prevent tables from being created

13. What does a database schema describe?

A. The current data stored in the database
B. The logical structure of the database
C. Only the primary keys
D. Only the foreign keys

What is the main difference between a database schema and a database instance?

A. The schema changes frequently, while the instance does not
B. The schema describes the structure, while the instance represents the current data
C. They are exactly the same
D. The instance describes attributes, while the schema describes tuples

15. A relation schema consists primarily of:

A. Tuples only
B. Attributes and their corresponding domains
C. Primary keys only
D. Foreign keys only

16. Which attribute can be used to relate the instructor and department relations?

A. Salary
B. Building
C. Dept name
D. Instructor name

17. What is the purpose of a key in a relational database?

A. To store multiple values in one attribute
B. To uniquely identify tuples
C. To sort all tuples alphabetically
D. To remove all null values

18. A superkey is:

A. A single attribute that must always be numeric
B. A set of attributes that uniquely identifies a tuple
C. A foreign key only
D. A key that cannot contain unnecessary attributes

19. Which of the following is a possible superkey for the instructor relation if ID is unique?

A. Name
B. Salary
C. ID
D. Dept name

20. Why is name generally not a superkey for the instructor relation?

A. Names cannot be stored in databases
B. Several instructors may have the same name
C. Names are always null
D. Names are foreign keys

21. A candidate key is:

A. Any set of attributes
B. A minimal superkey
C. Always a foreign key
D. A key containing unnecessary attributes

22. If {ID, name} is a superkey and ID alone is a candidate key, why is {ID, name} not a candidate key?

A. It contains a foreign key
B. It contains a null value
C. It is not minimal
D. It is not a valid attribute set

23. The candidate key selected by the database designer as the main identifier is called the:

A. Foreign key
B. Superkey
C. Primary key
D. Alternate domain

24. Which characteristic is desirable when selecting a primary key?

A. It should change frequently
B. It should contain duplicate values
C. Its values should be stable and rarely change
D. It should contain null values

25. What is a foreign key?

A. An attribute that uniquely identifies every relation
B. An attribute that references a key in another relation
C. An attribute that must always contain text
D. A key that cannot be used in another table

26. In the university database, dept name in the instructor relation can be a foreign key referencing:

A. course
B. student
C. department
D. section

27. The relation containing a foreign key is called the:

A. Referenced relation
B. Referencing relation
C. Primary relation
D. Candidate relation

28. What does referential integrity ensure?

A. Every table has exactly one attribute
B. Foreign-key values correspond to valid values in the referenced relation
C. All tuples are stored in sorted order
D. All attributes have the same domain

29. What is the main purpose of a schema diagram?

A. To display only the current database values
B. To graphically represent relations, attributes, and key dependencies
C. To replace the DBMS
D. To remove foreign keys

30. In a schema diagram, foreign-key dependencies are commonly represented by:

A. Circles
B. Arrows
C. Squares
D. Dotted tables

Chapter3: The Structure Query Language (SQL)

2.1 Introduction

A database is an organized collection of information about a particular entity or subject. It is designed to minimize unnecessary data redundancy while supporting multiple applications and users. A Database Management System (DBMS) is application software used to create, store, manage, and manipulate the data contained in a database. A query language provides a convenient way to access and retrieve data from a database. Structured Query Language (SQL) is the standard language widely used by relational database management systems (RDBMSs). SQL was developed by IBM in the late 1970s and has become the primary means of communication between clients and relational database servers.

Communication between a client and an RDBMS, or between two RDBMSs, is performed through SQL statements. The client may be a basic SQL interface or an application such as a graphical user interface (GUI), report generator, or another database system. The client sends SQL statements to the database server, which processes the requests and returns the results. This approach reduces network traffic because only the query request and its corresponding results need to be transmitted. Consequently, the client's processing resources can be used primarily for running the application.

SQL is a data sublanguage that includes three major components:

1. **Data Definition Language (DDL):** Used to define and modify the structure of database objects.

2. **Data Manipulation Language (DML):** Used to retrieve and manipulate data stored in the database.

3**. Data Control Language (DCL):** Used to control access to database objects and data.

SQL is considered a fourth-generation language (4GL) and provides many features for database definition, manipulation, and control. However, different commercial DBMS products may not implement all SQL features in exactly the same way. For example, Microsoft Access does not support some SQL features. In addition, certain SQL statements or constructs may not be portable between different database systems. Therefore, users should consult the documentation or help system of the specific DBMS to identify any implementation differences.

Most relational database systems support SQL, which facilitates the migration of databases and applications from one DBMS to another. This capability has contributed significantly to the widespread adoption and success of relational database management systems. When different DBMSs comply with the SQL standard, a user can develop applications that access data stored in multiple database systems. This capability is known as **portability**. However, individual RDBMS products may provide additional or customized SQL features that are not supported by other systems. Therefore, although there is a common SQL standard, specific implementations may differ between database systems.

SQL provides an interface through which users specify **what** information they want to retrieve or what operation they want to perform, without specifying **how** the operation should be executed. The DBMS is responsible for developing and optimizing the query execution plan. For example, when three tables, X, Y, and Z, need to be joined, the DBMS may choose either to execute (X JOIN Y) first and then join the result with Z, or to execute X JOIN (Y JOIN Z). The selection of the most efficient execution plan depends on factors such as database statistics, available indexes, and the size of the relations.

SQL is therefore described as a **non-procedural language**, because it specifies the required result or operation rather than the detailed steps needed to achieve it. In addition, SQL is a high-level language whose syntax is relatively close to natural English, making it easy to learn and use.

SQL has been standardized by the **American National Standards Institute (ANSI)** and other international standardization organizations. The first ANSI SQL standard was introduced in 1986 and was commonly referred to as **SQL-86** or **SQL-1**. It was subsequently revised and extended, resulting in standards such as **SQL-92 (SQL-2)**. Later standards introduced additional features and capabilities, including the standard commonly associated with **SQL:1999 (SQL-3)**. Modern SQL standards have continued to evolve, although particular DBMS products may implement additional features that are specific to their platforms.

2.2 Important Features of SQL

The major features of SQL include:

- **Non-procedural language:** SQL specifies what operation or result is required rather than describing the detailed procedure for achieving it.

- **English-like syntax:** SQL uses simple, readable commands that resemble natural English, making it relatively user-friendly.
- **Record and set processing:** SQL can operate on individual records as well as on sets of records.
- **High-level language:** Unlike traditional third-generation programming languages such as C and COBOL, SQL focuses on specifying what should be accomplished rather than how it should be accomplished.
- **Data sublanguage:** SQL includes several major components, particularly **Data Definition Language (DDL)**, **Data Manipulation Language (DML)**, and **Data Control Language (DCL)**.
- **Data abstraction:** SQL shields users from the internal database structures, storage mechanisms, and algorithms used by the DBMS.
- **Database views:** SQL provides mechanisms for creating and managing views that present selected portions of the database to users.
- **Security and authorization:** SQL supports mechanisms for controlling user access and protecting database resources.
- **Integrity constraints:** SQL enables the definition of rules that maintain the accuracy and consistency of stored data.
- **Transaction management:** SQL provides facilities for controlling database transactions and maintaining database consistency during concurrent operations.

2.3 Data Definition Language (DDL)

As discussed in the previous section, the basic storage structure of a relational database is a *table*, which organizes data into rows and columns. Each column represents a particular data field or attribute. The question then arises: **What type**

of data can be stored in a column, and how can the structure of a table be defined using SQL?

DDL is a set of SQL commands used to define, create, modify, and remove database schema objects such as tables, indexes, and views. DDL commands provide the functionality required to *create, alter, and drop* these database objects and are primarily concerned with the definition and administration of the database structure. DDL commands modify the database schema and, depending on the DBMS, their execution may cause the current transaction to be *implicitly committed*. This means that changes resulting from DDL operations are permanently recorded in the database and cannot normally be undone using a standard transaction rollback. The most commonly used DDL commands and their purposes are discussed in the following sections.

CREATE TABLE Command

Syntax:

CREATE TABLE <table name>(

Column_name1 data type (column width) [constraints], Column_name2 data type (column width) [constraints], Column_name3 data type (column width) [constraints],

……..

);

Where table name assigns the name of the table, column name defines the name of the field, data type specifies the data type for the field and column width allocates specified size to the field.

Guidelines for creation of table:

- Table name should start with an alphabet.
- In table name, blank spaces and single quotes are not allowed.
- Reserve words of that DBMS cannot be used as table name.
- Proper data types and size should be specified.
- Unique column name should be specified.

Column Constraints: NOT NULL, UNIQUE, PRIMARY KEY, CHECK, DEFAULT, REFERENCES,

On delete Cascade: Using this option whenever a parent row is deleted in a referenced table then all the corresponding child rows are deleted from the referencing table. This constraint is a form of referential integrity constraint.

Example 1:

```
CREATE TABLE product
(
pno number (4) PRIMARY KEY, pname char (20) NOT NULL,
qoh number (5) DEFAULT (100),
class char (1) NOT NULL,
rate number (8,2) NOT NULL,
```

```
CHECK ((class='A' AND rate<1000) OR
(class='B' AND rate>1000 AND rate<4500) OR (class='C' AND rate>4500))
);
```

The command above creates a table. Primary key constraint ensures that product number (pno) is not null and unique (both are the properties of primary key). Please note the use of data type char (20). In many implementations of SQL on commercial DBMS like SQL server and oracle, a data type called varchar and varchar2 is used respectively. Varchar basically is variable length character type subject to a maximum specified in the declarations. We will use them at most of the places later. Please note the use of check constraints in the table created above. It correlates two different attribute values.

Example 2:

```
CREATE TABLE prodtrans
(
pno number (4)
ptype char (1) CHECK (ptype in ('I','R','S')),
qty number (5)
FOREIGN KEY pno REFERENCES product (pno) ON DELETE CASCADE);
```

In the table so created please note the referential constraint on the foreign key **pno** in **prodtrans** table to **product** table. Any product record if deleted from the product table will trigger deletion of all the related records (ON DELETE CASCADE) in the **prodtrans** table.

ALTER TABLE Command: This command is used for modification of existing structure of the table in the following situation:

- When a new column is to be added to the table structure.

- When the existing column definition has to be changed, i.e., changing the width of the data type or the data type itself.
- When integrity constraints have to be included or dropped.
- When a constraint has to be enabled or disabled.

Syntax

ALTER TABLE <table name> ADD (<column name> <data type>…);

ALTER TABLE <table name> MODIFY (<column name> <data type>…);

ALTER TABLE <table name> ADD CONSTRAINT <constraint name> < constraint type>(field name);

ALTER TABLE <table name> DROP<constraint name>;

ALTER TABLE <table name> ENABLE/DISABLE <constraint name>;

You need to put many constraints such that the database integrity is not compromised.

Example 3:

ALTER TABLE emp MODIFY (empno NUMBER (7));

DROP TABLE Command:

When an existing object is not required for further use, it is always better to eliminate it from the database. To delete the existing object from the database the following command is used.

Syntax:

DROP TABLE<table name>;

Example 4:

DROP TABLE emp;

2.4 Data Manipulation Language (DML)

is a set of SQL commands used to retrieve, insert, modify, and delete data stored in existing database objects. Unlike DDL operations, DML statements generally do not cause an implicit commit. Therefore, changes made using DML statements are not permanently stored in the database until the transaction is explicitly committed. The main DML statements include:

SELECT: Retrieves data from one or more database tables.

INSERT: Adds new records to a table.

UPDATE: Modifies existing records in a table.

DELETE: Removes existing records from a table.

These commands enable users and applications to manipulate the data stored in the database while providing transaction control over the changes made.

1- SELECT Statement

This statement is used for retrieving information from the databases. It can be coupled with many clauses. Let us discuss these clauses in more detail:

1. Using Arithmetic operator

Example 5:

SELECT ENAME, SAL, SAL+300

FROM EMP;

2. Operator Precedence

The basic operators used in SQL are * / + -

Operators of the same priority are evaluated from Left to right Parentheses are used to force prioritized evaluation.

Example 6:

SELECT ENAME, SAL, 12*SAL+100

FROM EMP;

SELECT ENAME, SAL, 12*(SAL+100)

FROM EMP;

3. Using Column aliases

Example 7:

To print column names as NAME and ANNUAL SALARY SELECT ENAME "NAME", SAL*12 "ANNUAL SALARY"

FROM EMP;

4. Concatenation operator

Example 8:

Printing name and job as one string as column name employees:

```
SELECT ENAME||JOB "EMPLOYEES"
 FROM EMP;
```

5. **Using Literal Character String**

Example 9:

To print <name> IS A <job> as one string with column name employee

```
SELECT ENAME || ' IS A ' || JOB AS "EMPLOYEE"
FROM EMP;
```

6. **To eliminate duplicate rows (distinct operator)**

Example 10:

```
SELECT DISTINCT DEPTNO
FROM EMP;
```

7. **Special comparison operator used in where Clause**

a. between. …and…It gives range between two values (inclusive)

Example 11:

```
SELECT   ENAME,   SAL   FROM
EMP
WHERE SAL BETWEEN 1000 AND 1500;
```

b. In (list): match any of a list of values

Example 12:

```
SELECT EMPNO, ENAME, SAL, MGR FROM
EMP
WHERE MGR IN (7902, 7566, 7788);
```

7902, 7566, and 7788 are Employee numbers

c. Like: match a character pattern

- Like operator is used only with char and Varchar2 to match a pattern
- % Denotes zero or many characters
- _ Denotes one character
- Combination of % and_can also be used

Example 13:

(I) List the names of employees whose name starts with 's'

```
SELECT ENAME FROM EMP WHERE
ENAME LIKE 'S%';
```

(II) List the ename ending with 's' SELECT ENAME FROM EMP WHERE ENAME LIKE '%S';

(III) List ename having I as a second character

```
SELECT ENAME FROM EMP WHERE
ENAME LIKE '_I%';
```

d. Is null operator

Example 14:

to find employee whose manage-id is not specified

SELECT ENAME, MGR FROM EMP

WHERE MGR IS NULL;

e. Logical Operators

Rules of Precedence:

Order of Evaluation	Operator
1	All comparison operators (=, <>, >, <, >=, <=)
2	NOT
3	AND
4	OR

Example 15:

To print those records of salesman or president who are having salary above 15000/-

Select ename, job, sal from emp

Where job = 'SALESMAN' or job = 'PRESIDENT'

And sal>15000;

The query formulation as above is incorrect for the problem statement. The correct formulation would be:

SELECT ENAME, JOB, SAL FROM EMP

WHERE (JOB = 'SALESMAN' OR JOB = 'PRESIDENT') AND SAL>15000;

9. Order by clause

- It is used in the last portion of select statement

- By using this rows can be sorted
- By default it takes ascending order
- DESC: is used for sorting in descending order
- Sorting by column which is not in select list is possible
- Sorting by column Alias

Example 16:

SELECT EMPNO, ENAME, SAL*12 “ANNUAL” FROM EMP
ORDER BY ANNUAL;

Example 17:

Sorting by multiple columns; ascending order on department number and descending order of salary in each department.

SELECT ENAME, DEPTNO, SAL
FROM EMP
ORDER BY DEPTNO, SAL DESC;

10. Aggregate functions

- Some of these functions are count, min, max, avg.
- These functions help in getting consolidated information from a group of tuples.

Example 18:

Find the total number of employees.

SELECT COUNT(*) FROM EMP;

Example 19:

Find the minimum, maximum and average salaries of employees of department D1.

SELECT MIN(SAL), MAX(SAL), AVG(SAL) FROM EMP
WHERE DEPTNO = ‘D1’ ;

11. Group By clauses

- It is used to group database tuples on the basis of certain common attribute
- value such as employees of a department.
- WHERE clause still can be used, if needed.

Example 20:

Find department number and Number of Employees working in that department.

```
SELECT DEPTNO, COUNT(EMPNO)
FROM EMP
GROUP BY DEPTNO;
```

Please note that while using group by and aggregate functions the only attributes that can be put in select clause are the aggregated functions and the attributes that have been used for grouping the information. For example, in the example 20, we cannot put ENAME attribute in the SELECT clause as it will not have a distinct value for the group. Please note the group here is created on DEPTNO.

12. Having clause

- This clause is used for creating conditions on grouped information.

Example 21:

Find department number and maximum salary of those departments where maximum salary is more than Rs 20000/-.

```
SELECT DEPTNO, MAX(SAL) FROM
EMP
GROUP BY DEPTNO HAVING
MAX(SAL) > 20000;
```

INSERT INTO command:

- Values can be inserted for all columns or for the selected columns
- Values can be given through sub query explained in section 1.8
- In place of values parameter substitution can also be used with insert.
- If data is not available for all the columns, then the column list must be included following the table name.

Example 22:

Insert the employee numbers, an increment amount of Rs.500/- and the increment date-today (which is being entered through function SYSDATE) of all the managers into a table INCR (increment due table) from the employee file.

```
INSERT INTO INCR
SELECT EMPNO, 500, SYSDATE FROM EMP
WHERE JOB = 'MANAGER';
```

Example 23:

Insert values in a table using parameter substitution (& operator is used for it 1, 2 are the field numbers).

```
INSERT INTO EMP
VALUES (&1,'&2','&3', &4, &5, &6,NULL, &7); Please
```

note these values needs to be supplied at run time.

UPDATE Command:

Syntax is

UPDATE <table name>

SET <column name> = <value> WHERE

<condition>;

Sub query in the UPDATE command:

Example 24:

Double the commission for employees, who have got at least 2 increments.

```
UPDATE EMP
SET COMM = COMM * 2
WHERE 2 <= ( SELECT COUNT (*) FROM INCR
WHERE INCR.EMPNO = EMP.EMPNO
GROUP BY EMPNO);
```

Please note the use of subquery that counts the number of increments given to each employee stored in the INCR table. Please note the comparison, instead of ……>=2, we have written reverse of it as 2 <= …..

DELETE Command

In the following example, the deletion will be performed in EMP table.No deletion will be performed in the INCR table.

Example 25:

Delete the records of employees who have got no increment.

```
DELETE FROM EMP
WHERE EMPNO NOT IN (SELECT EMPNO FROM INCR);
```

Sheet 3

1) What are the advantages of SQL? Can you think of some disadvantages of SQL?

2) Create the Room, Booking, and Guest tables using the integrity enhancement features of SQL with the following constraints:

(a) Type must be one of Single, Double, or Family.

(b) Price must be between Rs.100/- and Rs.1000/-.

(c) roomNo must be between 1 and 100.

(d) booking dateFrom and dateTo must be greater than today's date.

(e) The same room cannot be double booked.

(f) The same guest cannot have overlapping bookings.

3) Define the function of each of the clauses in the SELECT statement. What are the restrictions imposed on these clauses?

4) Consider the supplier relations.

SNO	SNAME	STATUS	CITY

(Supplier Number)	(Supplier Name)		
S 1	P r e n t i c e H a l l	3	C a l c
S 2	M c G r a w H i l l	3 0	Mum
S 3	Wiley	2	C h e
S 4	P e a r s	4	D e l
S 5	G a l g o t	1	D e l

SNO	PNO (Part Number)	Quantity
S 1	P 1	3 0 0
S 1	P 2	2 0 0
S 2	P 1	1 0 0
S 2	P 2	4 0 0
S 3	P 2	2 0 0
S 4	P 2	2 0 0

a) Get supplier numbers for suppliers with status > 20 and city is Delhi

b) Get Supplier Numbers and status for suppliers in Delhi in descending order of status.

c) Get all pairs of supplier numbers such that the two suppliers are located in the same city. (Hint: It is retrieval involving join of a table with itself.)

d) Get unique supplier names for suppliers who supply part P2 without using IN operator.

e) Give the same query above by using the operator IN.

f) Get part numbers supplied by more than one supplier. (Hint : It is retrieval with sub-query block, with inter block reference and same table involved in both blocks)

g) Get supplier numbers for suppliers who are located in the same city as supplier S1. (Hint: Retrieval with sub query and unqualified comparison operator).

h) Get supplier names for suppliers who supply part P1. (Hint : Retrieval using EXISTS)

i) Get part numbers for parts whose quantity is greater than 200 or are currently supplied by S2. (Hint: It is a retrieval using union).

j) Suppose for the supplier S5 the value for status is NULL instead of 10. Get supplier numbers for suppliers greater than 25. (Hint: Retrieval using NULL).

k) Get unique supplier numbers supplying parts. (Hint: This query is using the built-in function count).

l) For each part supplied, get the part number and the total quantity supplied for that part. (Hint: The query using GROUP BY).

m) Get part numbers for all parts supplied by more than one supplier. (Hint: It is GROUP BY with HAVING).

n) For all those parts, for which the total quantity supplied is greater than 300, get the part number and the maximum quantity of the part supplied in a single supply. Order the result by descending part number within those maximum quantity values.

o) Double the status of all suppliers in Delhi. (Hint: UPDATE Operation). p) Let us consider the table TEMP has one column, called PNO. Enter into TEMP part numbers for all parts supplied by S2. q) Add part p7.

r) Delete all the suppliers in Mumbai and also the suppliers concerned.

Chapter4: Data Control

4.1 Introduction

Data Control refers to the SQL commands used to manage and control access to database resources. These commands allow database administrators to assign or revoke privileges and permissions for different users, thereby controlling which operations users are authorized to perform on database objects. Data Control Language (DCL) is primarily used for database security and access management. It helps ensure that users can access only the data and perform only the operations for which they have been authorized. The main DCL commands are **GRANT**, which is used to assign privileges to users, and **REVOKE**, which is used to withdraw previously granted privileges.

The following sections discuss the major Data Control Language commands in more detail.

Create a new user:

CREATE USER < user name > IDENTIFIED BY < Password>

Example 26:

CREATE USER MCA12 IDENTIFIED BY W123

Grant: It is used to provide database access permission to users. It is of two types (1) system level permission (2) Object level permission.

Example 27:

GRANT CREATE SESSION TO MCA12;

(This command provides system level permission on creating a session – not portable)

GRANT SELECT ON EMP TO MCA12;

(Object level permission on table EMP)

GRANT SELECT, INSERT, UPDATE, DELETE ON EMP TO MCA12;

GRANT SELECT, UPDATE ON EMP TO MCA12, MCA13;

(Two users)

GRANT ALL ON EMP TO PUBLIC;

(All permission to all users, do not use it. It is very dangerous for database)

Revoke: It is used to cancel the permission granted.

Example 28:

REVOKE ALL ON EMP FROM MCA12;

(All permissions will be cancelled)

You can also revoke only some of the permissions.

Drop: A user-id can be deleted by using drop command.

Example 29:

DROP USER MCA12;

Accessing information about permissions to all users

(1) Object level permissions: With the help of data dictionary you can view the permissions to user. Let us take the table name from oracle. In oracle the name of the table containing these permissions is user_tab_privs.

DESCRIBE USER_TAB_PRIVS ;

SELECT * FROM USER_TAB_PRIVS;

(2) System level permissions: With the help of data dictionary you can see them. Let us take the table name as user_sys_privs (used in oracle).

DESCRIBE USER_SYS_PRIVS ;

SELECT * FROM USER_SYS_PRIVS ;

All these commands are very specific to a data base system and may be different on different DBMS.

4.2 DATABASE OBJECTS: VIEWS, SEQUENCES, INDEXES AND SYNONYMS

Several important database objects are used to organize, manage, secure, and improve access to data. Among these objects are **views, indexes, sequences, and synonyms**. A **view** is a virtual representation of data derived from one or more database tables. Views can provide an additional level of **data abstraction and security** by allowing users to access only the required data without directly accessing the underlying tables. An **index** is a database object designed to improve the **speed and efficiency of data retrieval operations**. By providing an efficient method for locating records, indexes can significantly improve query performance, particularly when working with large tables.

Sequences are database objects used to generate a series of unique numeric values, which are commonly used for automatically generating identifiers or primary key values. **Synonyms** provide alternative names for database objects, making them easier to reference and, in some cases, hiding the actual object name or its location from users. The following sections discuss **views, sequences, indexes, and synonyms** in greater detail.

4.2.1 Views

A view is like a window through which data from tables can be viewed or changed. The table on which a view is based is called Base table. The view is stored as a SELECT statement in the data dictionary. When the user wants to retrieve data, using view. Then the following steps are followed.

1) View definition is retrieved from data dictionary table. For example, the view definitions in oracle are to be retrieved from table name USER-VIEWS.

2) Checks access privileges for the view base table.

3) Converts the view query into an equivalent operation on the underlying base table

Advantages:

- It restricts data access.
- It makes complex queries look easy.
- It allows data independence.

•It presents different views of the same data.

Table 4.1 Type of views: Simple views and Complex Views

Feature	Simple Views	Complex Views
Number of tables	One	One or more
Contain Functions	No	Yes
Contain groups of data	No	Yes
Data Manipulation	IS allowed	Not always

Let us look into some of the examples. To see all the details about existing views in Oracle:

SELECT* FROM USER_VIEWS; Creating a view:

- A query can be embedded within the CREATE VIEW STATEMENT
- A query can contain complex select statements including join, groups and sub-queries
- A query that defines the view cannot contain an order by clause.
- DML operation (delete/modify/add) cannot be applied if the view contains any of the following:

Delete (You can't delete if view contains following)	*Modify (you cannot modify if view contains following)*	*Insert (you cannot insert if view contains following)*
• Group functions • A group by clause • A distinct keyword	• Group functions • A group by clause • A distinct keyword • Columns defined by Expressions	• Group functions • A group by clause • A distinct keyword • Columns defined by Expressions • There are Not Null Columns in the base tables that are not selected by view.

Example 30: Create a view named employee salary having minimum, maximum and average salary for each department.

```
CREATE VIEW EMPSAL (NAME, MINSAL, MAXSAL, AVGSAL) AS
SELECT D.DNAME, MIN(E.SAL),MAX(E.SAL),AVG(E.SAL) FROM
EMP E, DEPT D

WHERE E.DEPTNO=D.DEPTNO

 GROUP BY D.DNAME;
```

To see the result of the command above you can give the following command:

```
SELECT * FROM EMPSAL;
```

You may get some sample output like:

```
NAME          MINSAL    MAXSA     AVGSAL ---------------          ---------
---------     ------------- ACCOUNTING       1300     5000        2916.6667

RESEARCH          800       3000             2175

SALES             950       2850            1566.6667
```

To see the structure of the view so created, you can give the following command:

```
DESCRIBE EMPSAL;

 Name                    Null?    Type

 ----------------- ------------    ---------------------

 NAME                    VARCHAR2 (14)

 MINSAL                   NUMBER
```

MAXSAL NUMBER

AVGSAL NUMBER

Creating views with check option: This option restricts those updates of data values that cause records to go off the view. The following example explains this in more detail:

Example 31:

To create a view for employees of Department = 30, such that user cannot change the department number from the view:

```
CREATE OR REPLACE VIEW EMPD30 AS
SELECT EMPNO EMPL_NUM, ENAME NAME, SAL SALARY
 FROM EMP
WHERE DEPTNO=30 WITH CHECK OPTION;
```

Now the user cannot change the department number of any record of view EMPD30. If this change is allowed then the record in which the change has been made will go off the view as the view is only for department-30. This restricted because of use of WITH CHECK OPTION clause

Creating views with Read only option: In the view definition this option is used to ensure that no DML operations can be performed on the view.

Creating views with Replace option: This option is used to change the definition of the view without dropping and recreating it or regranting object privileges previously granted on it.

1.6.2 Sequences

Sequences:

- automatically generate unique numbers
- are sharable
- are typically used to create a primary key value
- replace application code
- speed up the efficiency of accessing sequence Values when cached in memory.

Example 32:

Create a sequence named SEQSS that starts at 105, has a step of 1 and

can take maximum value as 2000.

```
CREATE SEQUENCE SEQSS START
WITH 105 INCREMENT BY 1
MAX VALUE 2000;
```

How the sequence so created is used? The following sequence of commands try to demonstrate the use of the sequence SEQSS.

Suppose a table person exists as:

```
SELECT * FROM PERSON;
```

Output:

CODE	NAME	ADDRESS
104	RAMESH	MUMBAI

Now, if we give the command:

INSERT INTO PERSON VALUES (**SEQSS.NEXTVAL**, &NAME, &ADDRESS)

On execution of statement above do the following input:

Enter value for name: 'Rakhi'

Enter value for address: 'New Delhi'

Now, give the following command to see the output:

SELECT * FROM PERSON;

CODE NAME ADDRESS --------- -

---------- ----------

104 RAMESH MUMBAI

105 Rakhi NEW DELHI

The descriptions of sequences such as minimum value, maximum value, step or increment are stored in the data dictionary. For example, in oracle it is stored in the table user_sequences. You can see the description of sequences by giving the SELECT command.

Gaps in sequence values can occur when:

- A rollback occurs that is when a statement changes are not accepted.
- The system crashes
- A sequence is used in another table.

Modify a sequence:

ALTER SEQUENCE SEQSS INCREMENT 2 MAXVALUE 3000;

Removing a sequence:

DROP SEQUENCE SEQSS; **1.6.3 Indexes and Synonyms**

Some of the basic properties of indexes are:

- An Index is a schema Object
- Indexes can be created explicitly or automatically
- Indexes are used to speed up the retrieval of rows

•Indexes are logically and physically independent of the table. It means they can be created or dropped at any time and have no effect on the base tables or other indexes.

•However, when a table is dropped corresponding indexes are also dropped.

Creation of Indexes

Automatically: When a primary key or Unique constraint is defined in a table definition then a unique index is created automatically.

Manually: User can create non-unique indexes on columns to speed up access time to rows.

Example 33:

The following commands create index on employee name and employee name + department number respectively.

CREATE INDEX EMP_ENAME_IDX ON EMP (ENAME);

CREATE INDEX EMP_MULTI_IDX ON EMP (ENAME, DEPTNO);

Finding details about created indexes: The data dictionary contains the name of index, table name and column names. For example, in Oracle a user-indexes and user-ind-columns view contains the details about user created indexes.

Remove an index from the data dictionary:

DROP INDEX EMP_ENAME_IDX;

Indexes cannot be modified. Synonyms

It permits short names or alternative names for objects. Example 34:

CREATE SYNONYM D30

FOR EMPD30; Now if we give command:

SELECT * FROM D30; The output will be:

NAME	MINSAL	MAXSAL	AVGSAL
ACCOUNTING	1300	5000	2916.6667
RESEARCH	800	3000	2175
SALES	950	2850	1566.6667

Removing a Synonym:

DROP SYNONYM D30;

4.3 Table Handling

In a Relational Database Management System (RDBMS), data from multiple tables can be retrieved and combined using a join operation. A join is a relational operation that combines two or more tables based on a common attribute or related domain, producing a single result table or view.

In SQL, a join can be specified by identifying the columns that contain related values. Traditionally, the matching condition between the tables can be specified in the WHERE clause. Two tables can be joined when they contain columns whose values belong to compatible domains and can therefore be used to establish a relationship between the tables.

The result of a join operation is a single table containing the selected columns from the participating tables. Each row in the result represents a combination of rows from the input tables for which the specified join condition is satisfied. In other words, rows from different tables are combined when the values of their related columns meet the required matching condition.

When joining multiple tables, an appropriate join condition is required to establish the relationships between the participating tables. For example, joining two tables requires at least one relationship condition between them. When three tables, X, Y, and Z, are joined, the conditions must establish the necessary relationships among the tables, such as X–Y and Y–Z or X–Z and Y–Z, depending on the database relationships and the required query.

SQL supports several types of join operations, each designed for a particular purpose. The four major types of joins discussed in this section are presented below.

(1) Equi Join: A join in which the joining condition is based on equality between values in the common columns. Common columns appear (redundantly) in the result table. Consider the following relations:

- customer (custid, custname, ……..) and
- order (custid, ordered,……..)

Example 35:

What are the names of all customers who have placed orders?

The required information is available in two tables, customer and order. The SQL solution requires joining the two table using equi join.

```
SELECT CUSTOMER.CUTOID, ORDER.CUSTOID,
CUSTONAME, ORDERID
FROM CUSTOMER, ORDER
WHERE CUSTOMER.CUSTOID=ORDER.CUSTOID;
```

The output may be:

Customer.custoid	order.custoid	custoname	orderid
10	10	Pooja Enterprises	1001
12	12	Estern Enterprises	1002
3	3	Impressions	1003

(2) **Natural Join**: It is the same like Equi join except one of the duplicate columns is eliminated in the result table. The natural join is the most commonly used form of join operation.

Example 36:

SELECT CUSTOMER.CUTOID, CUSTONAME, ORDERID FROM CUSTOMER, ORDER

WHERE CUSTOMER.CUSTOID=ORDER.CUSTOID;

Output:

custoid custoname orderid

---------- -------------------- ----------

10 Pooja Enterprises 1001

12 Estern Enterprises 1002

3 Impressions 1003

(3) **Outer Join**: The use of Outer Join is that it even joins those tuples that do not have matching values in common columns are also included in the result table. Outer join places null values in columns where there is not a match between Tables. A condition involving an outer join is that it cannot use the IN operator or cannot be linked to another condition by the OR operator.

Example 37: The following is an example of left outer join (which only considers the non-matching tuples of table on the left side of the join expression).

```
SELECT CUSTOMER.CUTOID, CUSTONAME, ORDERID
FROM CUSTOMER LEFT OUTER JOIN ORDER
WHERE CUSTOMER.CUSTOID =  ORDER.CUSTOID;
```

Output: The following result assumes a CUSTID in CUSTOMER table who have not issued any order so far.

CUSTOID	CUSTONAME	**ORDERID**
10	Pooja Enterprises	1001
12	Estern Enterprises	1002
3	Impressions	1003
15	South Enterprises	NULL

The other types of outer join are the Right outer join or complete outer join.

(4) **Self-Join**: It is a join operation where a table is joined with itself. Consider the following sample partial data of EMP table:

EMPNO	ENAME	MGRID	….
1 2 3 4 ….	Nirmal Kailash Vee	44 41 NU	

	n a B o s s … . .	L L …	

Example 38:

Find the name of each employee's manager name.

```
SELECT WORKER.ENAME || 'WORK FOR'
|| MANAGER.ENAME FROM EMP
WORKER, EMP MANAGER

WHERE
WORKER.MGR=MANAGER.E
MPNO; Output:
```

Nirmal works for Boss
Kailash works for Boss
Veena works for Nirmal

Sheet4

1. Which SQL command is used to grant privileges to a database user?

A. REVOKE

B. GRANT

C. DROP

D. ALTER

2. Which command is used to withdraw previously granted privileges?

A. DELETE

B. REMOVE

C. REVOKE

D. DROP

3. Which command creates a new database user in the examples provided?

A. CREATE USER

B. NEW USER

C. ADD USER

D. INSERT USER

4. Which statement grants the user MCA12 permission to retrieve data from the EMP table?

A. GRANT EMP TO MCA12;

B. GRANT SELECT ON EMP TO MCA12;

C. GRANT READ EMP TO MCA12;

D. GRANT SELECT EMP TO MCA12;

5. Which SQL command removes a database user?

A. DELETE USER

B. REMOVE USER

C. DROP USER

D. REVOKE USER

6. What is a database view?

A. A physical copy of an entire database

B. A virtual representation of data derived from tables

C. An index used for sorting

D. A command for granting privileges

7. Which is an advantage of using views?

A. They always increase storage requirements

B. They eliminate the need for base tables

C. They restrict data access

D. They automatically create sequences

8. A simple view is generally based on how many tables, according to the chapter?

A. One

B. Two

C. Three

D. Any number of tables

9. Which clause prevents an update through a view from moving a row outside the view's defined condition?

A. ORDER BY

B. GROUP BY

C. WITH CHECK OPTION

D. DISTINCT

10. Which view option prevents DML operations on the view?

A. WITH CHECK OPTION

B. READ ONLY

C. REPLACE

D. NEXTVAL

11. What is the primary purpose of a database sequence?

A. To generate numeric values automatically

B. To combine tables

C. To restrict user access

D. To rename tables

12. Which sequence attribute returns the next sequence value in the chapter's Oracle example?

A. SEQSS.CURRENT

B. SEQSS.NEXTVAL

C. SEQSS.VALUE

D. SEQSS.INCREMENT

13. Which of the following can cause gaps in sequence values?

A. Creating a view

B. Rolling back a transaction

C. Creating a synonym

D. Describing a table

14. What is the primary purpose of an index?

A. To grant user privileges

B. To create alternative object names

C. To speed up retrieval of rows

D. To generate unique numeric values only

15. When can a unique index be created automatically, according to the chapter?

A. When a synonym is created

B. When a primary key or UNIQUE constraint is defined

C. When a view is queried

D. When a user is dropped

16. What does a database synonym provide?

A. An alternative name for a database object

B. A copy of the table's data

C. A set of numeric values

D. A join condition

17. What is an equi join?

A. A join based on equality between column values

B. A join that always includes unmatched rows

C. A table joined with itself

D. A join that removes all duplicate rows

18. What distinguishes a natural join from an equi join in the chapter's explanation?

A. It never uses matching columns

B. It includes only unmatched rows

C. It eliminates one duplicate common column from the result

D. It joins a table only with itself

19. Which join includes unmatched rows and places NULL values in columns where no match exists?

A. Equi join

B. Outer join

C. Self-join

D. Natural join

20. Which join operation combines a table with itself?

A. Natural join

B. Left outer join

C. Equi join

D. Self-join

References:

1. **Seth White, Maydene Fisher, Jonathan Bruce**, JDBC API Tutorial and Reference, *2nd Edition, Addison-Wesley, 2002. Detailed reference for JDBC, ResultSet types, concurrency, and database operations.*
2. **Herbert Schildt**, *Java: The Complete Reference*, 11th Edition, McGraw-Hill Education, 2019. Comprehensive coverage of Java programming, OOP concepts, and standard libraries.
3. **R.Nageswara Rao**, *Core Java: An Integrated Approach*, 3rd Edition, Dreamtech Press, 2018. Focuses on Java fundamentals, classes, objects, inheritance, and exception handling.
4. **Patrick Niemeyer, Daniel Leuck**, *Java 8: The Complete Reference*, McGraw-Hill Education, 2015. Covers Java 8 features including functional programming and updated Java APIs.
5. **Oracle Java Documentation**, *Java SE Documentation*, https://docs.oracle.com/javase/ Official documentation for Java classes, JDBC, and core APIs.
6. https://www.javatpoint.info/java.html– Practical examples of Java programming and JDBC.